

8.5.1 证书

为了安全地分发公钥，第一种尝试的方法是我们想象有一个密钥分发中心（KDC, key distribution center）一天 24 小时地根据客户的需求提供在线公钥服务。这种方案有很多问题，其中一个问题是它的扩展性差，密钥分发中心将很快成为瓶颈。而且，如果它停止工作，则 Internet 安全性也将突然停顿下来。

由于这些原因，人们已经开发出了另一种不同的方案，这种方案并不要求密钥分发中心始终不停地保持在线。事实上，它并不需要时时在线。相反，它所要做的事情是证明每个公钥属于个人、公司或者其他组织。这种证明公钥所属权的组织现在称为认证中心（CA, Certification Authority）。

作为一个例子，假设 Bob 希望 Alice 和其他不认识的人能够安全地与他进行通信。他可以带上护照或者驾驶证到 CA 那里，请求认证他的公钥。然后，CA 给他颁发一个证书，其中的内容类似于图 8-24 所示的证书，而且 CA 用自己的私钥对证书的 SHA-1 散列值进行签名。然后，Bob 付给 CA 一定的费用，并获得一张包含了证书和签过名的散列值的光盘。

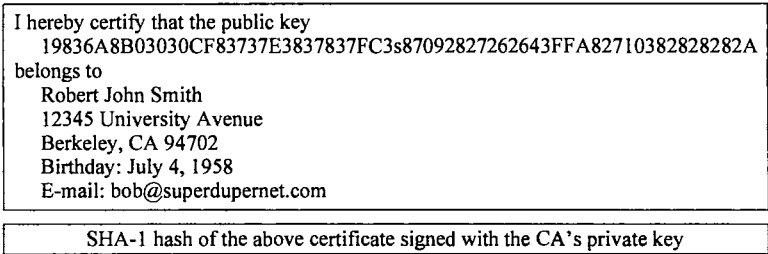


图 8-24 一个可能的证书和它的签名散列

证书的基本任务是将一个公钥与安全个体（个人、公司等）的名字绑定在一起。证书本身并不是保密的，也没有被保护。例如，Bob 可能决定将他的新证书放到 Web 站点上，他的做法是在主页上使用这样一个链接：“点击这里可得到我的公钥证书”。当用户单击该链接之后，就可以得到 Bob 的证书和签名块（即该证书经过签名之后的 SHA-1 散列值）。

现在我们再来看图 8-23 的场景。当 Trudy 截获了 Alice 请求 Bob 主页的命令时，她该怎么办呢？她可以把她自己的证书和签名块放到伪造的页面上，但是当 Alice 读取证书时，她立即就会发现那不是来自于 Bob，因为 Bob 的名字不在证书中。Trudy 可以临时修改 Bob 的主页，用她自己的公钥来替换 Bob 的公钥。然而，当 Alice 对证书运行 SHA-1 算法时，她得到的散列值与她将 CA 的公钥（这是众所周知的）应用在签名块上所得到的散列值不一致。由于 Trudy 拿不到 CA 的私钥，所以她无法生成一个包含她公钥的签名块，也就无法构造出让 Alice 相信她是 Bob 的 Web 页面。通过这种方式，Alice 可以确信她得到的是 Bob 的公钥，而不是 Trudy 或者其他人的公钥。正如前面我们所承诺的，这种方案不要求 CA 一直提供在线验证服务，从而消除了潜在的瓶颈问题。

虽然证书的标准功能是将一个公钥绑定到一个安全个体上，但证书也可以被用来将一个公钥绑定到一个属性（attribute）上。例如，一个公钥可以表达这样的含义：该公钥属于

某一个年龄超过 18 岁的人。这样的公钥可以被用来证明私钥的所有者是个成年人，从而被允许访问少儿不宜的资料等而不需要泄露所有者的身份。典型情况下，持有证书的人往往会把这个证书发送给网站、其他的安全个体，或者对年龄较为敏感的进程等。网站、安全个体或进程将生成一个随机数，并用证书中的公钥对随机数进行加密。如果所有者能够解密随机数并发送回来，那将证明所有者确实具备了该证书中所声明的属性。另外，此随机数也可以被用来生成一个会话密钥，以保证后续通话的安全。

在证书中可能包含属性的另一个例子是用在面向对象的分布式系统中。每个对象通常有多个方法。对象的所有者可以向每个顾客提供一个证书，在证书中给出一个位图来表明允许该顾客调用哪些方法，并且用一个签过名的证书将位图与一个公钥绑定在一起。同样地，如果证书持有者可以证明自己确实拥有相应的私钥，那么，他将能够执行位图中指定的方法。这个方法的特点是它不需要暴露拥有者的身份。这在个人隐私很重要的情形下非常有用。

8.5.2 X.509

如果每一个想要为某些资料做签名的人都到 CA 那里申请各种不同类型的证书，那么，管理所有不同格式的证书很快就成为一个问题。为了解决这个问题，ITU 已经设计并批准了一个专门针对证书格式的标准。该标准称为 **X.509**，现在它已经广泛应用于 Internet 上。自从 1988 年首次被标准化以来，它经历了三个版本。下面我们将讨论第 3 版（V3）。

X.509 深受 OSI 的影响，它借用了 OSI 领域中某些非常糟糕的特性（比如命名和编码）。然而，令人惊奇的是，在几乎所有其他的领域中，从机器地址到传输协议，再到电子邮件格式，IETF 一般都忽略 OSI，而努力按照自己的方式来做得更好，但是在证书格式上，IETF 却采纳了 X.509。X.509 的 IETF 版本由 RFC 5280 来描述。

X.509 的核心是提供一种描述证书的格式。图 8-25 列出了证书中的主要字段。这里给出的描述仅仅说明了每个字段的一般性用途。有关更多的信息，请参考标准本身或者 RFC 2459。

字段	含义
Version	X.509 的哪个版本
Serial number	此数值加上 CA 的名字唯一标识了证书
Signature algorithm	用来签名证书的算法
Issuer	CA 的 X.500 名字
Validity period	有效期的开始和终止时间
Subject name	哪个实体的密钥得到了验证
Public key	主题的公钥和所用算法的 ID
Issuer ID	可选的 ID，唯一标识了证书颁发者
Subject ID	可选的 ID，唯一标识了证书主题
Extensions	已经定义了多少扩展字段
Signature	证书的签名（用 CA 的私钥加密）

图 8-25 X.509 证书的基本字段

例如，如果 Bob 在 Money 银行的信贷部门工作，那么，他的 X.500 地址可能是这样的：

/C=US/O=MoneyBank/OU=Loan/CN=Bob/

其中 C 代表国家，O 代表组织，OU 代表组织单位，CN 是一个普通名字。CA 和其他

的实体也用类似的方式来命名。关于 X.500 名字的一个实质成问题是，如果 Alice 试图通过 bob@moneybank.com 与 Bob 联系，而她拿到的证书中却是 X.500 名字，那么，对于 Alice 来说，该证书是否引用了她想要的那个 Bob 却并不那么显而易见。幸运的是，从第 3 版开始，它也允许使用 DNS 名字来代替 X.500 名字，所以，这个问题最终可能被消除了。

证书的编码使用了 OSI 的抽象语法标记 1 (ASN.1, Abstract Syntax Notation 1)，你可以将这种编码想象成与 C 语言结构类似的描述形式，除此以外它也包含一些非常特殊和细微的标记。有关 X.509 更多的信息，请参考 (Ford 和 Baum, 2000)。

8.5.3 公钥基础设施

由一个 CA 来颁发全世界所有的证书显然是不切实际的。它将会不堪重负，并且成为一个中心故障点。一种可能的解决方案是使用多个 CA，并且所有的 CA 由同一个组织来运行，它们使用同一个私钥来签名证书。虽然这样可以解决负载太重和单故障点的问题，但是它又引入了新的问题：密钥泄漏。如果有几十台服务器遍布在世界各地，并且所有的服务器都持有 CA 的私钥，那么私钥被偷窃或者泄漏的机会就会明显地增加。由于该私钥的暴露将会影响到全世界的电子安全基础设施，所以，采用一个中心 CA 的做法是非常危险的。

另外，由哪一个组织负责运行这个 CA 呢？很难想象有哪一个权威机构能在全球范围内被普遍接受和信任。在有些国家中，人们坚持这个组织应该是一个政府机构，而在其他的国家中，人们又坚持这个组织不应该属于一个政府机构。

基于这些理由，人们又发展了另一种证明公钥身份的方法。它的通用名称是公开密钥基础设施 (PKI, Public Key Infrastructure)。在这一小节中，我们将一般性地概述一下 PKI 的工作原理，不过，由于有许多关于 PKI 的提案和建议，所以有些细节将来可能会发生变化。

一个 PKI 有多个组件，包括用户、CA、证书和目录。PKI 所做的事情是提供一种方法将这些组件有序地组织起来，并且定义了各种文档和协议的标准。一种特别简单的 PKI 形式是 CA 层次，如图 8-26 所示。在这个例子中，我们显示了三个层次，但实际上，层次的数目可能更少，也可能更多。最顶级的 CA，即层次的根，它的责任是证明第二级的 CA，这种 CA 我们称为区域管理机构 (RA, Regional Authority)，因为它们可能覆盖某一个地理区域，比如一个国家或者一个洲。然而，这个术语不是标准的；事实上，PKI 并没有定义标准的术语来表达树结构的不同层次。这些区域管理机构又依次证明下一级 CA 的真实性；在图中所示的 PKI 结构中，这些下级 CA 才真正为组织和个人颁发 X.509 证书。当根 CA 授权一个新的 RA 时，它生成一个 X.509 证书，并且在证书中声明它已经批准了这个 RA，同时将该 RA 的公钥也包含在这个证书中；根 CA 为证书签过名以后，将它交给该 RA。类似地，当 RA 批准一个新 CA 时，它产生一个证书并为其签名；该证书声明了它已经批准该 CA，并包含 CA 的公钥。

我们的 PKI 按照如下方式工作。假设 Alice 需要 Bob 的公钥以便与他通信，她搜寻并找到了一个包含 Bob 公钥的证书，该证书已由 CA 5 签过名。但是，Alice 从来没有听说过 CA 5。她所知道的是 CA 5 也许是 Bob 的 10 岁的女儿。她可以到 CA 5 那里说：请证明你

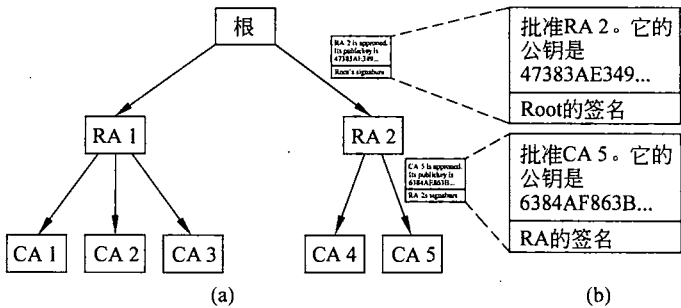


图 8-26
(a) 一个层次 PKI; (b) 一个证书链

的合法性。CA 5 用它从 RA 2 那里获得的证书作为响应，该证书包含了 CA 5 的公钥。现在 Alice 拿到了 CA 5 的公钥，所以她可以验证 Bob 的证书确实是由 CA 5 签名的，因此是合法的。

除非 RA 2 是 Bob 的 12 岁儿子，否则接下来的步骤是，Alice 请 RA 2 证明它的合法性。RA 2 对此的应答是一个由根 CA 签名的证书，其中包含了 RA 2 的公钥。现在 Alice 可以确信她真正拿到了 Bob 的公钥。

但是，Alice 如何找到根 CA 的公钥呢？这正是 PKI 的奥妙所在。它假设每个人都知道根 CA 的公钥。例如，她的浏览器可能在发行的时候已经预装了根的公钥。

Bob 是一个非常友好的合作伙伴，他不想给 Alice 增加太多额外的工作。他知道她将要检查 CA 5 和 RA 2 的合法性，所以，为了省去她的麻烦，他把两个必要的证书收集起来，并将这两个证书连同他自己的证书一起交给 Alice。现在，Alice 可以使用她自己所了解到的根证书来验证顶级证书和其中所包含的公钥，进而再验证第二级证书。按照这种方法，Alice 不用跟任何人联系就可以完成验证工作。因为证书全都是经过签名的，所以它可以检测出所有对证书内容的篡改行为。像这样由底向上回溯到根的证书链有时候称为信任链（chain of trust），或者证书路径（certification path）。这项技术已经得到广泛的应用。

当然，我们还有另一个问题，即谁来运行根 CA 呢？解决方案是，并非只有一个根，而是有多个根，每个根有它自己的 RA 和 CA。实际上，现代浏览器在安装的时候预先加载了 100 多个根的公钥，有时候这些根称为信任锚（trust anchor）。通过这种方式，就可以避免在全球范围内使用单一的可信权威机构。

但是，现在又引出了另一个问题，即浏览器厂商如何确定哪些声称的信任锚是可靠的，哪些是不可靠的。对于用户来说，归根到底只能信任浏览器厂商会做出明智的选择，相信它们不会简单地把那些愿意支付包含费的根 CA 全部变成信任锚。大多数浏览器允许用户检查根密钥（通常以证书的形式出现，并且包含根自身的签名），也允许用户删除掉那些看起来不太让人放心的根密钥。

目录

对于任何一个 PKI，另一个问题是在哪里存放证书（以及能回溯到某一个知名信任锚的证书链）。一种可能的方案是让每个用户保存他（她）自己的证书。虽然这样做很安全（即

用户不可能在篡改了签过名的证书以后不会被检测到),但是并不方便。提出来的另一种方案是将 DNS 做成一个证书目录。Alice 在与 Bob 联系之前,她或许要通过 DNS 来查找 Bob 的 IP 地址,所以,为什么不让 DNS 在返回 IP 地址的时候也返回 Bob 的完整证书链呢?

有些人认为这是正确的解决之道,但是,其他人则希望使用专门的目录服务器来管理 X.509 证书。这样的目录允许用户利用 X.509 名字的属性来提交查询任务。例如,在理论上,这样的目录服务可以回答诸如这样的问题:“将所有在美国或者加拿大销售部门工作的名为 Alice 的人的名单交给我。”

撤销

现实世界中也充满了各种各样的证书,比如护照和驾驶证。有时候这些证书可以被撤销,例如,对于酒后驾车或者其他的违章驾驶行为,警察局有权吊销驾驶员的驾驶证。在数字世界中也有同样的问题:一个证书的颁发者可能因为证书持有者以某种方式滥用他的证书而决定撤销他的证书。如果拥有者的私钥已被泄漏,或者更糟糕的情况下,CA 的私钥被泄漏,那么,这时候相关的证书都要被撤销。因此,任何一个 PKI 都需要处理有关证书撤销的事宜。撤销的可能性加剧了事情处理的复杂化。

处理的第一步是让每个 CA 定期地发布一个证书撤销列表(CRL, Certificate Revocation List),该 CRL 列出了所有已被撤销的证书序列号。由于证书包含了过期时间,所以 CRL 只需包含那些尚未过期的证书的序列号。一旦证书的过期时间已经过去,则该证书自动失效,所以,在那些已经过期的证书与真正被撤销的证书之间并不需要做专门的区分。无论哪一种情形,这些证书都不能再被使用了。

不幸的是,引入 CRL 意味着一个用户在使用证书之前必须先要获得 CA 的 CRL,以便确定该证书是否已被撤销。如果它确实已被撤销,则用户就不应该再使用该证书。然而,即使该证书不在 CRL 列表中,它也有可能在 CA 发布 CRL 列表之后刚刚被撤销。因此,真正有保证的唯一途径是询问 CA。而且,当下次再使用这个证书时,用户必须再次询问 CA,因为该证书有可能在几秒钟之前刚刚被撤销掉。

另一个复杂之处在于一个已撤销的证书也可以被恢复为有效证书,例如,如果撤销的原因仅仅是因为用户没有支付必要的费用。这种必须处理证书的撤销(可能也要处理恢复)要求同时也消除了证书的最佳特性之一,即用户在使用证书的时候可以不必与 CA 联系。

那么,CRL 应该被保存在哪里呢?一个理想的选择是证书本身所在的地方。一种策略是,让 CA 主动地定期推出 CRL,然后由各个目录服务器对 CRL 进行处理,它们只需删除那些被撤销的证书即可。如果目录服务器本身不保存证书,那么可以将 CRL 缓存在网络中的各个便利之处。由于 CRL 本身也是一个被签过名的文档,所以,如果它被篡改的话,用户很容易检测出来。

如果证书的有效期很长,那么 CRL 也将很长。例如,如果信用卡的有效期为 5 年,则尚在有效期内而被撤销的信用卡的数量,一定比每隔 3 个月发行新卡的情形要多得多。处理这种长 CRL 的一种标准方法是每过一段(较长)时间发行一个全列表,但在这期间频繁地发行 CRL 更新消息。这样做可以减少因分发 CRL 而需要的带宽。

8.6 通信安全

现在我们已经结束了对各种交易工具的学习。大多数重要的技术和协议也都已经涉及了。本章剩余部分将讨论如何把这些技术应用到实践中来提供网络安全性，同时在本章末尾还将提到一些有关安全性之社会因素的思考。

在接下来的4小节中，我们将讨论通信安全，也就是说，如何将数据位秘密地、未被篡改地从源端传送到接收方，以及如何将有害的数据位排除在外。这些问题绝不是网络环境中唯一的安全问题，但它们无疑是最重要的，所以，通信安全是一个很好的学习起点。

8.6.1 IPSec

多年以来 IETF 一直很清楚，Internet 缺乏安全性。要在 Internet 上增加安全性并不容易，因为曾经爆发过一场关于在哪里加入安全机制的争论。绝大多数安全专家相信，为了真正做到安全，加密和完整性检查必须是端到端的（即位于应用层上）。这就是说，源进程对数据进行加密，以及/或者实施完整性保护，然后将数据发送给目标进程；在目标进程中，数据被解密，以及/或者被验证其完整性。若企图在这两个进程之间（包括在操作系统内部）篡改数据，则必定可以被检测到。这种做法的麻烦在于，它要求改变所有的应用系统，以便它们能够感知到安全特性的存在。从这个角度来说，次好的方法是将加密功能放到传输层上，或者放到应用层与传输层之间的一个新层上，这样仍然可以做到端到端的安全，但是不要求改变应用系统。

另一种相反的观点是，用户并不理解安全性，他们不具备正确地使用它的能力，并且没有人愿意以任何一种方式来修改已有的程序，所以网络层应该认证和/或者加密数据包，而不应该让用户卷入进来。在经过多年的激烈争论之后，这种观点赢得了足够的支持，因此 IETF 定义了一个网络层的安全标准。这么做的部分理由是，在网络层上实施加密操作，并没有妨碍那些了解安全性的用户正确地使用这些安全特性，而且多多少少可以帮助那些对安全性一无所知的用户。

这场争战的结果是提出了一个称为 **IP 安全 (IPSec, IP security)** 的设计方案，它由 RFC 2401、2402 和 2406 等一些 RFC 文档描述。然而，并不是所有的用户都想要加密功能（因为加密操作的计算代价较高）。虽然 IPSec 并没有将加密功能做成可选项，相反，它总是要求加密功能，但它允许使用空算法（null algorithm）。空算法因其简单、易于实现和极高的速度而获得了高度赞赏，RFC 2410 描述了空算法。

完整的 IPSec 设计方案是一个多服务、多算法和多粒度的框架。IPSec 支持多种服务的原因在于考虑了并不是每个人都愿意为所有的服务、所有的时间支付费用，所以，IPSec 按照点菜的方式来提供这些服务。最主要的服务是保密性、数据完整性，以及针对重放攻击（即入侵者重放一次会话过程）的保护。所有这些服务都建立在对称密钥密码学的基础之上，因为高性能在这里非常关键。

IPSec 支持多算法的考虑在于任何一个现在被认为安全的算法在将来某个时候可能会

被攻破。将 IPSec 设计成与算法无关的好处是，即使某个特殊的算法将来被打破了，这个框架仍然可以幸存下来。

IPSec 支持多粒度的理由是，这样可使得它既能够保护单个 TCP 连接，也能够保护一对主机之间的所有流量，或者一对安全路由器之间的所有流量，以及其他一些可能性。

关于 IPSec，一个略微使人惊讶的方面在于，尽管它落在 IP 层，但它是面向连接的。实际上，这也无须惊讶，因为为了做到任何一种安全性，建立一个密钥并且在一定长的时间内使用该密钥——在本质上，这就是一种连接，只是用了不同的名称。而且，在面向连接的环境中，建立连接的开销可以被分摊到许多数据包。在 IPSec 环境中，一个“连接”称为一个安全关联（SA，security association）。SA 是两个端点之间的单纯连接，它有一个与之关联的安全标识符。如果两个方向上都需要安全通信的话，则要求使用两个安全关联。数据包携带着安全标识符通过这些安全连接，而标识符是用来查询密钥和其他有关安全信息。

技术上，IPSec 有两个主要部分。第一部分描述了两个新的头，这两个新的头可以被加入到数据包中以便携带安全标识符、完整性控制数据和其他的信息。另一部分是 Internet 安全关联及密钥管理协议（ISAKMP，Internet Security Association and Key Management Protocol），处理建立密钥。ISAKMP 是个框架，用来开展工作的主要协议是 Internet 密钥交换（IKE，Internet Key Exchange）。由于早期版本的 IKE 存在严重的缺陷，（Perlman 和 Kaufman，2000）指出应该使用在 RFC 4306 中描述的 IKE 第 2 版本。

IPSec 有两种使用模式。在传输模式（transport mode）中，IPSec 头被直接插在 IP 头的后面。IP 头中的 Protocol 字段要做相应的修改，以表明有一个 IPSec 头紧跟在普通 IP 头的后面（但是在 TCP 头的前面）。IPSec 头包含了安全信息，主要是 SA 标识符、一个新的序号，可能还包括有效载荷数据的完整性检查信息。

在隧道模式（tunnel mode）中，整个 IP 数据包，连同头和所有的数据一起被封装到一个新的 IP 数据包的数据部分，并且这个 IP 数据包有一个全新的 IP 头。当隧道的终点并不是最终的目标节点时，隧道模式将非常有用。在有些情况，隧道的终点是一台安全网关机器，例如，公司的一个防火墙。在这种模式中，当数据包通过防火墙时，防火墙负责封装或者拆封数据包。由于隧道终止于这台安全的机器，所以公司 LAN 上的机器不必知晓 IPSec 的存在。只有防火墙必须要知道 IPSec。

当一束 TCP 连接被聚合起来，作为一个加密的流被处理的时候，隧道模式也非常有用，这种用法的好处是，入侵者将无法看到谁给谁发送了多少数据包。有时候，仅仅知道有多少流量发送到哪里，这本身就是很有价值的信息。例如，如果在一次军事危机中，五角大楼和白宫之间的网络流量突然减少了许多，但是五角大楼与科罗拉多州洛矶山脉（Colorado Rocky Mountains）某个军事基地之间的流量却增加了同等的数量，那么入侵者就有可能从这些数据中推断出某些有用的信息。研究数据包的流模式（即使是加密的数据流）称为流量分析（traffic analysis）。IPSec 的隧道模式在某种程度上提供了一种应对的方法。隧道模式的缺点是，它加入了一个额外的 IP 头，因此实实在在地增加了数据包的长度。与此相反，IPsec 的传输模式不会明显影响数据包的长度。

第一个新的头是认证头（AH，Authentication Header）。它提供了完整性检查和防止重放攻击的安全性，而没有提供保密性（即没有加密数据）。在传输模式中 AH 的用法如

图 8-27 所示。在 IPv4 中，AH 被插在 IP 头（包括可能有的所有选项）和 TCP 头之间；在 IPv6 中，它仅仅是另一个扩展头而已，就如同一般的扩展头那样。实际上，AH 的格式与标准的 IPv6 扩展头的格式非常接近。有效载荷部分有可能必须被填充到某个特殊的长度以适应认证算法的要求，如图 8-27 所示。

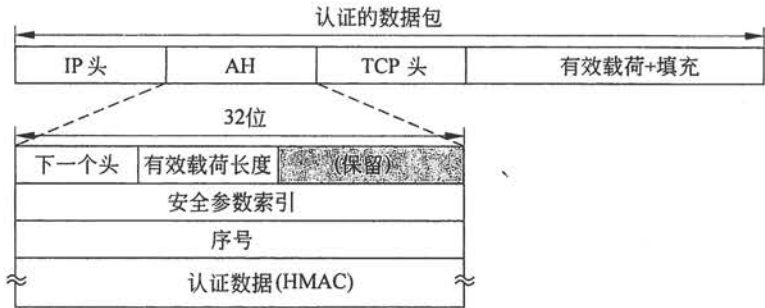


图 8-27 IPv4 传输模式下的 IPSec 认证头

现在我们来考查 AH 头。下一个头（Next header）字段被用来保存原始的 IP 包的协议（Protocol）字段，该字段后来被替换成 51 以表明后面紧跟着一个 AH 头。在绝大多数情况下，这里出现的是 TCP 协议代码（6）。有效载荷长度（Payload length）字段等于 AH 头部中 32 位字的个数减 2。

安全参数索引（Security parameters index）是连接标识符。它由发送方插入，用来指定接收方数据库中一条特定的记录。该记录包含了这个连接上所使用的共享密钥，以及有关该连接的其他信息。如果这个协议是由 ITU 而不是 IETF 发明的，则这个字段称为虚拟电路号（virtual circuit number）。

序号（Sequence number）字段用来对通过一个 SA 发送的所有数据包进行编号。每个数据包都有一个唯一的编号，即使重传的数据包也有唯一的编号。换句话说，重传的数据包与原始的数据包具有不同的序号（尽管它们的 TCP 序号是相同的）。这个字段的目的是为了检测重放攻击。这些序号不会发生回绕的现象，因为如果 2^{32} 个序列号全部用完以后；IPSec 必须建立一个新的 SA 以便继续进行通信。

最后是 Authentication data（认证数据）字段，它是一个可变长度字段，包含了有效载荷的数字签名。当 SA 被建立时，双方协商将使用哪个签名算法。通常情况下，这里不使用公开密钥密码学，因为数据包的处理速度必须极快，而所有已知的公钥算法都太慢。由于 IPSec 建立在对称密钥的基础上，而且发送方和接收方在建立 SA 之前协商了一个共享密钥，所以，在计算签名过程中可以利用这个共享密钥。一种简单的办法是在数据包和共享密钥合起来的位串上计算散列值。当然，这个共享的密钥并不在网络上传输。像这样的方案称为散列的消息认证码（HMAC，Hashed Message Authentication Code）。它的计算速度比先运行 SHA-1 然后在其结果上再运行 RSA 要快得多。

AH 头并不支持数据加密功能。因此，只有在需要完整性检查但不需要保密性的场合，AH 头才十分有用。AH 头有一个值得注意的特性，那就是它的完整性检查也覆盖了 IP 头中的某些字段，即当数据包在路由器之间被转发时不随路由器而变化的那些域。例如，TTL（Time to live）字段在每一跳上都要改变，所以它不能被包含在完整性检查的范围内。然而，

IP 源地址被包含在检查范围内，这就使入侵者无法伪造数据包的来源信息。

另一个 IPSec 头是封装的安全有效载荷 (ESP, Encapsulating Security Payload)。图 8-28 显示了 ESP 在传输模式和隧道模式下的使用情形。

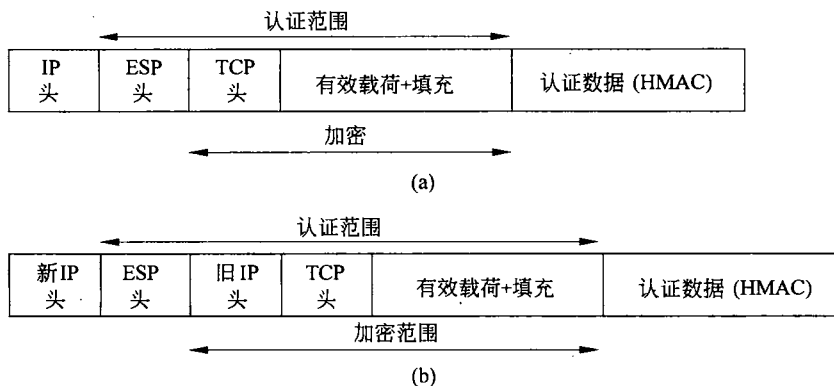


图 8-28

(a) 传输模式中的 ESP; (b) 隧道模式中的 ESP

ESP 头由两个 32 位字组成。它们是安全参数索引 (Security parameters index) 和序号 (Sequence number) 字段，我们在 AH 头中已经看到过这两个字段了。通常跟在这两个字后面的第三个字是用于数据加密的初始向量 (Initialization vector)，如果使用空加密算法，则可以省略这个字段。

如同 AH 一样，ESP 也提供了 HMAC 完整性检查，但是完整性检查的范围并不包含头部，而是在有效载荷之后，如图 8-28 所示。将 HMAC 放在末尾对于硬件实现是有好处的。在数据位通过网络接口卡往外发送的过程中，同时可以计算出 HMAC，然后再追加到尾部。这也正是为什么以太网和其他的 LAN 将它们的 CRC (循环冗余校验码) 放在尾部而不是头部的原因。若使用 AH，则数据包必须被缓冲，并且在发送数据包之前首先要计算出签名，这样有可能会降低每秒钟发送的数据包的数量。

既然 ESP 可以完成 AH 所能做的任何事情，甚至还能做得更多，而且效率也更高，那么，一个很自然产生的问题是：为什么还要引入 AH 呢？这主要是由于历史的原因。最初，AH 只处理完整性，而 ESP 只处理保密性。后来，完整性被加入到 ESP 中，但是，那些设计 AH 的人不希望在 AH 已经实用之后让它死掉。然而，他们唯一提出的真正论据是，AH 检查 IP 头的一部分，而 ESP 不检查 IP 头，当然这并没有多少说服力。另一个也没有什么说服力的原因是，如果一个产品支持 AH 但不支持 ESP，那么它或许可以省却有关出口许可的诸多麻烦，因为它不能够做加密操作。AH 在将来有可能被慢慢淘汰掉。

8.6.2 防火墙

能够将任何地方的任何一台计算机与另外一个地方的另一台计算机连接起来，这种能力令人喜忧参半。对于家庭个人用户来说，在 Internet 上漫游充满了各种乐趣。但对于公司的安全管理员来说，这是一种梦魇。大多数公司有大量的保密信息放在网络上，例如交易的秘密、产品开发计划、市场策略、财务分析等。如果这些信息被泄露给竞争对手将会

带来可怕的后果。

除了信息往外泄漏的危险性以外，同时还存在着信息渗透进来的危险性。尤其是，病毒、蠕虫和其他的数字有害物可能会破坏安全性、销毁有用的数据，管理员必须消耗大量精力才能清理它们留下的混乱局面。这些有害物通常是由于粗心的员工在玩各种新奇的游戏时引入的。

因此，公司的网络需要有一些机制允许“好的”数据位进来，而阻止“坏的”数据位出去。一种方法是使用 IPSec。这种方法可以保护安全站点之间传送的数据。然而，IPSec 并不能阻止数字有害物和入侵者进入到公司的 LAN 中。为了看清楚如何实现这个目标，我们需要考察防火墙。

防火墙（firewall）只不过是现代版的古代中世纪安全设施，即在城堡周围挖一条很深的护城河来保护城堡。在古代，这种设计强迫每个进出城堡的人都要经过一座吊桥，而他们在通过吊桥时，哨兵可以对他们进行检查。在网络环境中，也可以采用相同的方法。公司可以按照任意的方式将多个 LAN 连接起来，但是强迫所有进出公司的流量必须通过一座电子吊桥（即防火墙），如图 8-29 所示。除此之外，没有其他路由可走。

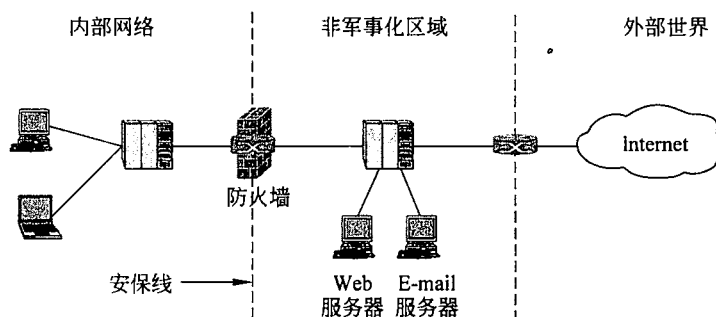


图 8-29 保护内部网络的防火墙

防火墙的作用就像一个包过滤器（packet filter）。它检查进出的每个数据包。通常只有那些符合网络管理员所定规则和条件的数据包才被放行，而那些不符合条件的数据包则被丢弃。

典型的过滤标准以规则或者表格列出，它通常含有可接受的源点和目标点，被阻塞的源点和目标点，以及含有关于往来其他机器的数据包的处理规范。在通常的 TCP/IP 设置中，源或者目标点也许都含有一个 IP 地址和一个端口。端口号说明了哪些服务是需要的。例如，在 TCP 中，端口 25 用于电子邮件，端口 80 用于 HTTP。可以简单地通过阻塞某些端口来过滤数据包。例如，一个公司可以阻塞所有 IP 地址上带有 TCP 端口 79 的数据包。曾经很普遍的一个做法是，让 Finger 服务搜寻人们的电子邮件地址，但是这种方法现今已经很少采用。

其他的端口并不是那样容易被封杀。困难在于网络管理人员既要保证网络安全，但又不能切断与外界的通信。要设置得让网络比较安全并不难，但是要使得没有人抱怨并不容易。这就是图 8-29 中非军事化区域（DMZ，DeMilitarized Zone）的设置派上用场的地方。DMZ 是公司网络的一部分，但是它被放置在安全装置（例如防火墙）的外部，是进入网络的入口。放置一个诸如 Web 服务器那样的机器在 DMZ 中，Internet 上的计算机可以联系它

来浏览公司的网页。现在可以设置防火墙使之阻塞所有想通过端口 80 的入境 TCP 流量，从而 Internet 上的计算机无法通过这个端口来攻击在公司内部网的计算机。为了允许对 Web 服务器实施管理，可以在防火墙上制定一个允许 Web 服务器和内部机器连接的规则。

通过一段时期跟攻击者的竞争，防火墙已经变得非常复杂。刚开始时，防火墙只是将一组规则独立地用在每个数据包，但是实践已经证明很难制定一些规则，它既能让有用的信息通过而又能封杀所有不想要的流量。状态防火墙（Stateful firewalls）将数据包映射到连接，并且使用 TCP/IP 的头字段来保持跟踪连接。这样的设置倒是可以制定出一些规范，例如允许一个外部的 Web 服务器给内部机器发送数据包，但是仅在内部机器先跟外部 Web 服务器建立一个连接之后。这样一个规则不可能用于无状态设计，因为它只能使所有的来自外部 Web 服务器的数据包要么全部通过要么全部丢弃。

来自状态处理的另一层复杂性是为防火墙实现应用层的网关（application-level gateway）。这个处理涉及防火墙要检查数据包内部（甚至至 TCP 头后部分）来查看应用正在做什么。具备了这样的能力，防火墙就有可能区别用于网页浏览的 HTTP 流量和用于对等文件分享的 HTTP 流量。管理员可以制定规则避免公司被用于对等文件共享，但同时又保证对公司运行至关重要的网页浏览。所有这些方法，不仅要检查出境流量还要检查入境流量，例如，为了防止公司内部的敏感信息通过电子邮件方式泄露出去。

经过上述讨论，我们应该很清楚防火墙违反了协议的标准分层。它们是网络层设备，但是它们偷窥传输层和应用层内容来做过滤。这使它们变得脆弱。例如，防火墙往往依赖于标准端口号的约定来判断数据包含有哪类流量。虽然标准端口常被按约定使用，但是并非所有的应用都使用标准端口。一些对等应用动态地选用标准端口来避免被轻易地认出（和阻塞）。使用 IPSEC 或者其他加密手段使得防火墙无法摄取上层信息。最后，当计算机相互交谈通过防火墙时，防火墙无法告诉它们使用的规则，以及为何要撤销他们的连接。它只能简单地假装线路不通。基于所有这些原因，网络纯粹主义者认为防火墙是 Internet 体系结构上的一个污点。然而，对计算机来说 Internet 可能是一个危险的地方，防火墙能帮助它们避免遭受攻击，所以还是不会被淘汰。

即使防火墙的配置极其完美，网络中仍然会存在大量的安全问题。例如，如果一个防火墙被配置成只允许接收来自某些特定网络的数据包（比如，来自公司其他分部的流量），那么，防火墙外面的入侵者可以在数据包中填上假的源地址，从而绕过防火墙的检查机制。如果一个内部人员想要偷取保密文档，他可以加密这些文档，或者先把文档拍摄下来然后以 JPEG 文件的形式将图片转运出来，这样可以绕过任何电子邮件过滤器。我们一直没有提到，事实上，在所有的攻击事件中，虽然有四分之三来自防火墙外部，但是来自防火墙内部的攻击通常损害最大，例如来自心怀不满的员工报复（Verizon, 2009）。

防火墙的另一个不同问题是它提供了单道防线来阻挡攻击，如果它被攻破就全盘皆输。基于这个原因，通常都使用多层防火墙来防范攻击。例如，一个防火墙可以放置在内部网入口，而且每个计算机可能也要运行自己的防火墙。那些认为只需要一道安全关口就足够的读者，显然近来没有搭乘过定时国际航班。

另外，还存在许多其他类型的攻击是防火墙无法处理的。防火墙的基本思想是阻止入侵者进入，或者避免保密数据被转运出去。不幸的是，还有一些人的手段更加低劣，他们不会别的，只会想方设法把目标站点搞瘫痪。他们的做法是，向目标机器发送大量合法的

包，直至目标机器不堪重负而崩溃。例如，为了削弱一个 Web 站点，入侵者可以发送一个 TCP SYN 包来请求建立一个连接。然后 Web 站点为这个连接分配一个内部表项，并且发送一个 SYN+ACK 包作为应答。如果入侵者不再响应的话，则这个表项将被占用一定长的时间（比如几秒钟），直至它超时。如果入侵者在短时间内发送数千个连接请求，则所有的表项都将被占用，从而 Web 站点无法再建立起正常合法的连接。在这种攻击中，入侵者的意图是使目标机器停止服务，而不是偷取数据，因此这种攻击称为**拒绝服务攻击**（DoS, Denial of Service）。一般情况下，请求包中的源地址是伪造的，所以入侵者不太容易被追踪到。DoS 攻击 Web 站点的现象在 Internet 很常见。

一个更加糟糕的变种是，入侵者已经攻破了遍布世界各地的数百台计算机，然后他命令所有这些机器同时向同一个目标发起攻击。这种方法不仅增加了入侵者的威力，而且也减小了他被检测到的几率，因为那些攻击包来自大量的机器，并且这些机器属于普通的可信用户。这样的攻击称为**分布式拒绝服务**（DDoS, Distributed Denial of Service）。这种攻击很难防护。即使被攻击的机器能够快速识别出伪造的请求，它也必须花一定的时间来处理该请求，然后再将它丢弃；如果每秒钟到达的请求足够多，CPU 将不得不花费所有的时间来处理这种请求。

8.6.3 虚拟专用网络

许多公司的办事处和部门分散在多个城市中，有时候甚至分布在多个国家。过去，在公共数据网络出现以前，对于这些公司来说，最常见的做法是从电话公司租用一些线路，将各场所或者全部场所两两连接起来。现在有些公司仍然这样做。利用公司的计算机和租用的电话线路而建立起来的网络称为**专用网络**（private network）。

专用网络工作得很好，而且也非常安全。如果可用的线路仅仅是租用线路的话，则所有的流量不会泄漏到公司各工作场所以外，入侵者必须通过物理搭线的方法才能闯入，而这种物理搭线的方法并不容易做到。但是，这种专用网络存在的问题是，租用一条 T1 线路的开销是每个月几千美元，而租用一条 T3 线路可能需要几倍的价格。当公共数据网络以及后来的 Internet 出现以后，许多公司希望将它们的数据流量（可能还有语音流量）转移到公共网络上，但是又不希望损失专用网络的安全性。

这种需求很快导致了**虚拟专用网络**（VPN, Virtual Private Networks）的诞生。VPN 是建立在公共网络之上的层叠网络，但是具有专用网络的绝大多数特性。它们之所以称为“虚拟的”，是因为它们仅仅是一个假想的网络，就好像虚电路并不是真正的电路、虚拟内存并不是真正的内存一样。

一个流行的做法是直接 Internet 上构建 VPN。一个很常见的设计是在每个办事处都设置一个防火墙然后在所有办事处之间创建一条通过 Internet 的隧道，如图 8-30(a)所示。使用 Internet 连接的进一步优势是隧道可以根据需要而设置，比如包含一个员工在家里的计算机，或者他正在旅行时用的笔记本电脑，只要他能够有 Internet 连接就可以。这种灵活性远远优于使用租用的线路。然而站在 VPN 的角度来看，拓扑结构看起来就像专用网络一样，如图 8-30(b) 所示。当系统启动时，每一对防火墙必须协商它们的 SA 参数，包括服务、模式、算法和密钥。如果隧道采用了 IPSec，那么很有可能要聚合两个办事处之间的所

有流量组成一个经过认证的加密 SA，从而提供完整性控制、保密性，甚至可考虑流量分析的防护措施。许多防火墙有内置的 VPN 功能。一些常规路由器也有这功能。但是因为防火墙主要用在安全方面，很自然防火墙上有了隧道的起点和隧道的终点，它在公司和 Internet 之间提供了一条清楚的分界线。因此，防火墙、VPN 和具有 ESP 的 IPSec（隧道模式）是很自然的一种组合，并且在实践中得到了广泛应用。

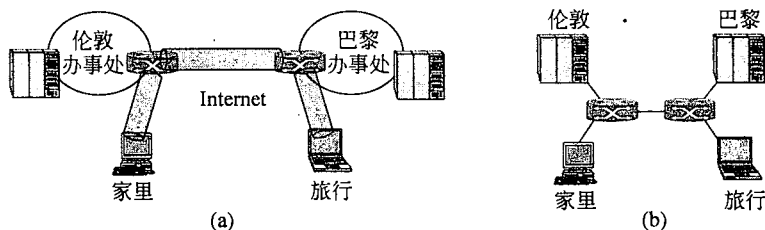


图 8-30

(a) 虚拟专用网络；(b) 从内部看的拓扑结构

一旦建立起 SA，数据包流量就可以开始流动了。对于 Internet 上的一台路由器而言，沿着 VPN 隧道传递的一个数据包仅仅是一个普通的数据包而已。这个数据包的唯一不同寻常之处在于 IPSec 头出现在 IP 头的后面，但是，由于这个额外的头不影响转发过程，所以，路由器并不关心这个额外的 IPSec 头。

另一个逐渐流行的方法是让 ISP 来设置 VPN。使用 MPLS（正如第 5 章讨论的），VPN 流量的路径甚至可以穿过公司办事处之间的 ISP 网络。这些路径保持使得 VPN 流量独立于 Internet 上其他流量，并且可以获得一定数量的带宽或者其他的服务质量保障。

VPN 的一个关键好处是它对于所有的用户软件都是透明的。防火墙建立并管理 SA。唯一知道这种配置方案的人是系统管理员（他必须配置和管理安全网关），或者是 ISP 管理员（他必须配置 MPLS 路径）。对于其他人来说，这样的网络与租用线路的专用网络并无区别。有关 VPN 的更多信息，请参考（Lewis，2006）。

8.6.4 无线安全性

利用 VPN 和防火墙来设计一个逻辑上绝对安全的系统是出奇地容易，但是，实际上，这样的系统仍然有可能像筛子一样漏洞百出。例如，如果有些机器是无线的，它们使用无线电波进行通信，这样就会在进入和出去两个方向上绕过防火墙。802.11 网络的范围通常是几百米距离，所以，如果一个人想要暗中监视一家公司，那么他只需早晨开车到该公司的停车场，并把一台支持 802.11 协议的笔记本电脑留在汽车里以便记录下所有听到的内容，然后他自己离开停车场。到下午晚些时候，该计算机的硬盘上就充满了各种有价值的信息。理论上，这种泄漏是不应该发生的。同样地，理论上，人们是不应该去抢劫银行的。

大多数安全问题的根源在于无线基站（接入点）的生产商，它们总是试图使自己的产品对用户极为友好。通常情况下，如果用户将设备从包装盒中取出来并插到电源插口上，它立即就开始运行了——它几乎没有任何安全性，所有秘密都暴露给无线电波范围内的任何一个人。如果它后来又被插入到一个以太网中，则所有的以太网流量也突然出现在停车场内。无线连接使得偷窥者的梦想成为现实：无须做任何工作就可以免费获得数据。因此，

毋庸多说，安全性对于无线系统比有线系统更加重要。在这一小节中，我们将考查无线网络处理安全性的几种方法。要想获得更多的信息，请参考（Nichols 和 Lekkas, 2002）。

802.11 安全性

部分 802.11 标准，最初称为 802.11i 规定了一个无线数据链路层安全协议，用来防止一个无线节点阅读或干扰另一对无线节点之间发送的消息。它还有个商标名称为 **WiFi 保护接入 2**（WPA2, WiFi Protected Access 2）。纯 WPA 是一个临时方案，它实现了 802.11i 的一个子集。应该避免使用它，而使用 WPA2。

我们马上将介绍 802.11i，在那之前我们首先应该知道它是有线等效保密（WEP, Wired Equivalent Privacy）的替代品，这是第一代 802.11 安全协议。WEP 由一个网络标准委员会设计，这个过程与 NIST 选取 AES 设计的过程完全不同。结果是灾难性的。问题究竟出在何处？从安全角度来看，它几乎每个方面都有问题。例如，WEP 加密数据使用了和流密码输出一样的异或操作。不幸的是，弱密钥分配意味着输出经常被重复使用。由此导致了通过不断尝试就可以攻破它的保护的直接后果。另一方面，它的完整性检查基于 32 位的 CRC。这种方式对于检测传输错误是足够的，但它并不是一个可用来防范攻击的很强保密机制。

这些以及其他设计上的缺陷使 WEP 很容易被攻破。第一个攻破 WEP 的实际演示由 Adam Stubblefield 在 AT&T 当实习生时完成（Stubblefield 等，2002），他能够在一个星期内编写和测试 Fluhrer 等描述的攻击，而且他把多数时间花在说服管理人员买一个实验用的无线 WiFi 卡。现在一分钟内破解 WEP 的软件唾手可得并且是免费的，因此极不倡导使用 WEP。虽然它可以防止那些随意的访问，但它并不能提供任何形式的真正安全保证。了解到 WEP 被严重破解后，802.11i 组仓促上马，于 2004 年 6 月提出了一个正式的标准。

现在我们来介绍 802.11i，如果适当地设置和使用，它能够提供真正的安全防护。WPA2 通常用在两种常见的场合下。第一种情形是企业设置，在这种情况下公司有一台单独的认证服务器，该服务器上有一个用户名和口令数据库，根据数据库来确定一个无线用户是否允许访问网络。在这种设置中，客户端使用标准的协议向网络验证他们自己。主要的标准是 802.1X，接入点用该协议让客户端和认证服务器对话并观察其结果，而可扩展的身份验证协议（EAP, Extensible Authentication Protocol）（RFC 3748）则告诉了客户端和认证服务器是如何互动的。事实上，EAP 是个框架，其他标准定义了协议消息。然而，我们不会深入到交互的很多细节，因为这些内容对于概述并不重要。

第二种情形是在家庭设置，此时没有认证服务器。相反，只有一个被客户端用来访问无线网络的共享密码。这种设置比起认证服务器要简单得多，这也是为何它通常被用在家庭环境或者小型公司。但相比之下它的安全性也弱了一些。两种设置的主要区别在于有认证服务器时，每个用户都拿到一个用来加密流量的密钥，而其他用户对此并不知晓。使用单个共享密码时，虽然它为每个客户端推导出不同的密钥，但所有的客户端具有相同的密码，只要愿意，客户端们也能推导出其他客户端的密钥。

用来加密流量的密钥是在身份验证的握手过程中计算出来的。握手过程发生在客户和一个无线网络关联，并且与一个认证服务器认证后产生的。在握手开始时，客户端要么有一个共享的网络密码，要么有一个认证服务器的密码。这个密码被用来推导出一个主密钥。然而，该主密钥并没有直接被用来加密数据包。实际上应该采用标准的加密方法来推导出

每次使用期限内的会话密钥、修改不同会话的密钥并尽可能少地暴露主密钥。

如图 8-31 所示, 会话密钥是通过四次包 (four-Packet) 握手计算出来的。首先, 接入点 (AP) 发出一个随机数用作识别。像这样在安全协议中仅使用一次的随机数称为临时值 (nonce), 它或多或少是 “number used once” (一次性数字) 的一个简写。客户可以选取它自己的临时值, 然后它用临时值、它的 MAC 地址和 AP 的 MAC 地址以及主密钥作为参数计算出一个会话密钥 K_S 。会话密钥由几部分组成, 每一部分都有不同的用途, 但我们忽略这个细节。现在客户有会话密钥, 但接入点 (AP) 还没有。因此客户发送它的临时值给 AP, AP 执行相同的计算推导出同样的会话密钥。临时值可以明文发送, 因为在缺乏额外机密信息的情况下, 会话密钥无法从临时值中推导出来。从客户发出的消息用消息完整性检查 (MIC, Message Integrity Check) 来加以完整性保护。AP 在它计算会话密钥之后检查 MIC 是否正确, 以便确定消息的确来自于该客户。就像 HMAC 一样, MIC 是消息认证码的另一个名字。因为很有可能跟 MAC (Medium Access Control) 地址混淆, MIC 术语通常用来替代网络协议。

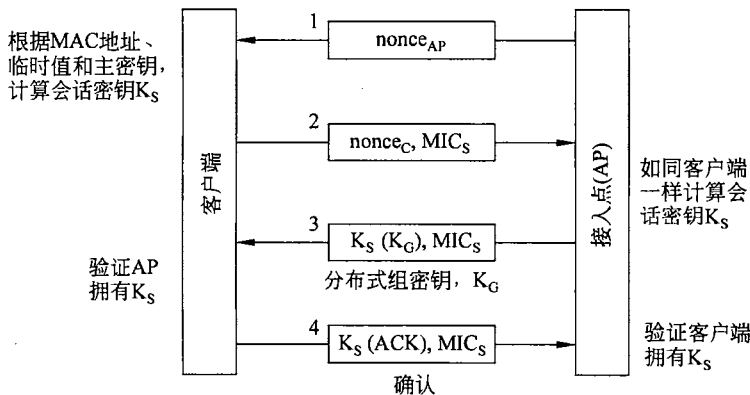


图 8-31 802.11i 密钥建立的握手过程

在最后两个消息中, AP 分发一个组密钥 K_G 给客户, 并且客户要确认该消息。这些消息的接收让客户验证 AP 有正确的会话密钥, 反之, AP 也知道客户端有正确的密钥。组密钥用在 802.11 局域网中广播和组播流量。因为四次包握手的结果是每个客户都有自己的加密密钥, 因此 AP 无法使用这些密钥中的任何一个给所有的无线客户广播数据包; AP 必须为每个客户发送一个单独的副本, 并且使用该客户的密钥。相反, 给所有的无线客户分发一个共享密钥可以使得广播流量只发送一次, 并且被所有的客户端接收。当有客户离开或者加入网络时, 该共享密钥必须被更新。

最后, 我们讨论如何实际使用密钥来提供安全性。802.11i 可使用两个协议来提供消息的保密性、完整性和身份验证。其中一个协议称为临时密钥完整性协议 (TKIP, Temporary Key Integrity Protocol), 它像 WPA 一样, 也是一个临时解决方案。它被设计用来改进老和慢的 802.11 网卡安全性, 因此在某些安全性上面至少比 WEP 更好, 它可以被当成固件升级。然而, 它现在也被破解了, 所以你最好用另一个推荐的协议 CCMP。究竟什么是 CCMP? CCMP 是一个蔚为壮观的名称缩写, 它的全称是具有密码块链的计数器模式消息认证码协议 (Counter mode with Cipher block chaining Message authentication code Protocol), 我们就简称它为 CCMP, 你可以随意地称呼它。CCMP 的工作原理很简单。它用 AES 加密, 使用

的密钥和块大小都是 128 位。它的密钥来自于会话密钥。为了提供保密性，消息用计数器模式的 AES 加密。记得我们曾在 8.2.3 节讨论过密码模式（Cipher mode），这些模式防止了每次用相同的位来加密相同的消息。计数器模式在加密中混合了计数值。为了提供完整性，包括头字段在内的消息都用密码块链模式进行了加密，并且最后长度为 128 位的块作为 MIC 被保存起来。然后，消息（用计数器模式加密后）和 MIC 同时被发送出去。在接收到一个无线数据包时，客户和 AP 各自执行加密操作，或者验证这个加密消息。对于广播或者组播的消息，使用共享的组密钥执行相同的处理步骤。

蓝牙安全性

蓝牙（Bluetooth）的无线范围比 802.11 要短得多，所以它不容易受到来自停车场的攻击，但是它的安全性仍然是个问题。例如，想象 Alice 的计算机配备了一个无线蓝牙键盘。在缺少安全防护的情况下，如果 Trudy 碰巧在邻近的办公室，那么她就可以读取到 Alice 敲入的所有信息，包括她往外发送的所有电子邮件。她也可以捕获到 Alice 计算机发送给不远处蓝牙打印机的所有信息（比如 Alice 接收到的电子邮件或者机密报告）。幸运的是，蓝牙有一个非常精致的安全方案，它试图让全世界 Trudy 们的攻击无法得逞。现在我们概要地描述这个安全方案的主要特性。

蓝牙版本 2.1 和更新的版本有 4 种安全模式，从“没有任何安全性”到“完全的数据加密和完整性控制”。如同 802.11 一样，如果安全功能被禁止（默认模式），则没有任何安全性可言。大多数用户关闭了安全功能；直到发生了严重的安全事件以后他们才将安全功能打开。在农业社会中，这种做法相当于等到马跑掉以后，才想起来要锁上马房的门。

蓝牙在多个层上提供了安全性。在物理层，跳频机制提供了一定程度的安全性，但是由于任何一个蓝牙设备在进入到一个微网时，它必须要被告知跳频序列，所以这个序列很显然不是一个秘密。当一个新到来的从节点向主节点请求一个信道时，真正的安全性开始了。在蓝牙 2.1 之前，假设两个设备分享一个预设置的密钥。在有些情况下，两个设备是由同一个生产商搭配在一起的（比如，头戴耳机和移动电话配套出售）。在其他情况下，一个设备（比如头戴耳机）有一个硬置的密钥，用户必须将这个密钥（十进制数）输入到另一个设备中（比如移动电话）。这些共享的密钥称为万能密钥（passkey）。不幸的是，万能密钥通常被预先设置为“1234”或者另一个可以猜测到的值，总之是仅允许有 10^4 种选择的 4 位数字。在蓝牙 2.1 中，使用简单的安全配对，设备选取 6 位数字。虽然它让万能密钥更难被猜到，但是距离安全还是太遥远。

为了建立一条信道，从节点和主节点都要检查对方，看它是否知道万能密钥。如果知道，则它们之间进行协商以便确定信道是否要加密、是否要进行完整性控制，或者两者都要。然后它们选择一个随机的 128 位会话密钥，其中有些位可能是公开的。之所以允许弱化这个密钥，是为了遵从不同国家的政府限制，从而避免出口或者使用超出政府破解能力的密钥长度。

蓝牙的加密操作使用了一个称为 E_0 的流密码；完整性控制使用 SAFER+。两者都是传统的对称密钥块密码算法。SAFER+ 曾经参加了 AES 算法竞赛，但是第一轮就被淘汰了，因为它比其他候选算法都要慢。蓝牙在 AES 密码被选定之前就已经确定选择 SAFER+，否则它极有可能会使用 Rijndael。

使用流密码的实际加密过程如图 8-14 所示, 其中明文与密钥流做 XOR 操作以便生成密文。不幸的是, E_0 本身(就像 RC4 一样)可能有致命的弱点(Jakobsson 和 Wetzel, 2001)。虽然在本书撰写时它还没有被攻破, 但是它与 A5/1 密码之间的相似之处引起了人们的关注(Biryukov 等, 2000), A5/1 的惨败危及所有的 GSM 电话流量。有时候人们(包括我在内)真的很惊讶, 在密码编码学家和密码分析家之间长期的“猫与老鼠”的游戏中, 密码分析家总是赢家。

另一个安全问题是蓝牙只认证设备, 而不认证用户, 所以蓝牙设备被盗之后, 窃贼有可能访问到用户的金融账户或者其他账户。然而, 蓝牙在上层实现了安全性, 所以即使链路层的安全性被突破, 其他的安全性仍然存在, 尤其是对于那些要求从某种键盘上手工敲入 PIN 码才能完成交易的应用来说更是如此。

8.7 认证协议

认证(Authentication)是这样的一项技术: 一个进程通过认证过程来验证它的通信对方是否是它所期望的实体而不是假冒者。在面对一个恶意的主动入侵者时, 要验证远程进程的身份是极其困难的, 它要求使用一些基于密码学理论的复杂协议。在本节中, 我们将学习许多认证协议, 这些协议可被用在不安全的计算机网络中。

顺便提一下, 有些人可能会混淆授权(authorization)和认证(authentication)这两个概念。认证所针对的问题是, 你是否真的在跟一个特定的进程通信; 而授权关注的是允许这个进程做什么样的事情。例如, 一个客户进程与文件服务器建立联系, 并且说: 我是 Scott 的进程, 我想删除文件 `cookbook.old`。从文件服务器的角度来看, 有两个问题是它必须要回答的:

- (1) 这真的是 Scott 的进程吗(认证)?
- (2) Scott 是否有权删除 `cookbook.old`(授权)?

只有当这两个问题都得到明确肯定的答复之后, 客户所请求的动作才可以执行。前一个问题实际上是一个非常关键的问题。一旦服务器知道了它在跟谁通话, 那么, 检查授权就非常容易了, 它只要在本地的数据表或者数据库中查找一些记录即可。由于这个原因, 我们在本节中把注意力集中在认证上。

基本上所有的认证协议都使用一个通用的模型, 如下所述。Alice 首先发起认证过程, 她给 Bob 或者可信的密钥分发中心(KDC, Key Distribution Center)发送一条消息。假设这里的 KDC 是诚实可靠的。接下来在不同方向上继续交换其他一些消息。当这些消息被发送出去以后, Trudy 可能截取、修改或者重放这些消息以便欺骗 Alice 和 Bob, 或者纯粹捣乱他们的工作。

然而, 当协议完成的时候, Alice 确信她是在跟 Bob 通话, 而 Bob 也确信他是在跟 Alice 通话。而且, 在绝大多数协议中, 两者还将建立起一个秘密的会话密钥(session key), 用在接下来的会话过程中。实际上, 尽管公开密钥密码算法被广泛应用于认证协议本身, 同时也被用于建立会话密钥, 但出于性能上的考虑, 所有的数据流量都是用对称密钥密码算法(通常为 DES 和 AES)加密的。

为每个新连接选择一个新的随机选取的会话密钥，这样做的好处是最小化用户密钥或者公钥来发送的流量，从而减少了入侵者可能得到的密文数量，而且，如果进程崩溃或者核心转储落入敌手也可将损失降到最小。期望达到的效果是发生意外事件后唯一被暴露的密钥是会话密钥。当会话建立起来以后，所有的永久密钥应该谨慎地退出通信用过程。

8.7.1 基于共享密钥的认证

在第一个认证协议中，我们假设 Alice 和 Bob 已经共享了一个密钥 K_{AB} 。这个密钥有可能是通过电话或者面对面地建立起来的，但无论如何，肯定不是在（不安全的）网络上建立的。

这个协议以下面一个在许多其他的认证协议中都出现的原理为基础：一方给另一方发送一个随机数，后者将这个随机数做一个特殊的替代，再把结果返回给前者。这样的协议称为质询-回应（challenge-response）协议。在本小节的协议以及后续的认证协议中，我们将使用下面的记号：

- A, B 是 Alice 和 Bob 的标识。
- R_i 是质询，其中下标指明了发起挑战的一方。
- K_i 是密钥，这里 i 代表密钥的所有者。
- K_S 是会话密钥。

我们的第一个共享密钥认证协议中的消息序列如图 8-32 所示。在消息 1 中，Alice 以一种 Bob 能理解的方式将她的标识发送给 Bob。当然，Bob 无法判断这条消息来自 Alice 还是 Trudy，所以他选择发送一个质询，即一个大的随机数 R_B ，并且它以明文方式发给 Alice，如图中的消息 2。然后 Alice 用她与 Bob 共享的密钥加密此消息，并且在消息 3 中将密文 $K_{AB}(R_B)$ 发送回来。当 Bob 看到这条消息时，他立即知道这条消息来自于 Alice，因为 Trudy 并不知道 K_{AB} ，因此她不可能生成此消息。而且，由于 R_B 是在一个很大的空间中被随机选取的（比如说是一个 128 位的随机数），所以 Trudy 不太可能曾经在以前的会话中看到过 R_B 和相应的回应。同样地，她也不可能猜测到任何一个质询随机数所对应的正确回应。

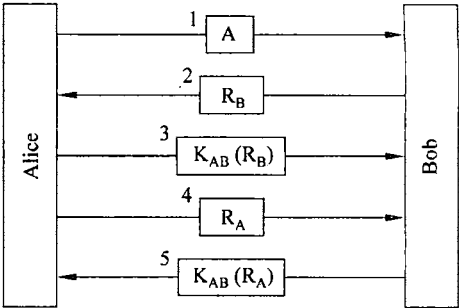


图 8-32 使用“质询-回应”的双向认证

这时候，Bob 已经确信与他通话的是 Alice，但是 Alice 还不能确定对方的身份。Alice 知道，Trudy 可能已经截取了消息 1 并送回了 R_B 作为回应。也许 Bob 昨天晚上已经死了。为了搞清楚与她通话的到底是谁，Alice 选取了一个随机数 R_A ，并以明文方式送给 Bob，见图中的消息 4。当 Bob 以 $K_{AB}(R_A)$ 作为回应的时候，Alice 知道她在跟 Bob 通话。如果他们现在希望建立一个会话密钥，则 Alice 可以选取一个 K_S ，并且用 K_{AB} 加密之后发送给 Bob 即可。

图 8-32 中的协议包含了 5 条消息，我们来看是否可以让它更为精简一些，以便省下几条消息。图 8-33 显示了一种做法。在这里，Alice 主动发起了质询-回应协议，而不是等 Bob

发起。类似地，当 Bob 回应 Alice 的质询时，他也发送自己的质询消息。整个协议可以减少为 3 条消息，而不再是 5 条消息。

这个新的协议是否比原来的协议有所改进呢？从某种意义上讲，确实是因为新协议更短了。不幸的是，它也是错误的。在特定的条件下，Trudy 利用一种称为反射攻击（reflection attack）的技术可以使该协议失败。尤其是，如果 Trudy 能够与 Bob 一次打开多个会话，则 Trudy 就可以攻破这个协议。这种条件是有可能成立的，比如，Bob 是一家银行，他随时准备接受来自柜台机器的多个并发连接。

Trudy 的反射攻击如图 8-34 所示。首先，Trudy 声称她是 Alice，并且发送 R_T ，然后 Bob 如同往常一样，用他自己的质询 R_B 作为回应。现在 Trudy 被定住了，她该怎么办呢？她并不知道 $K_{AB}(R_B)$ 。

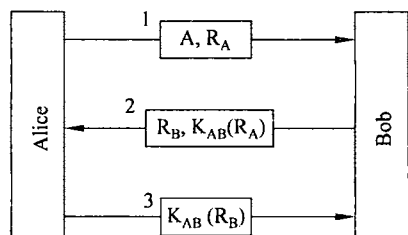


图 8-33 一个缩短的双向认证协议

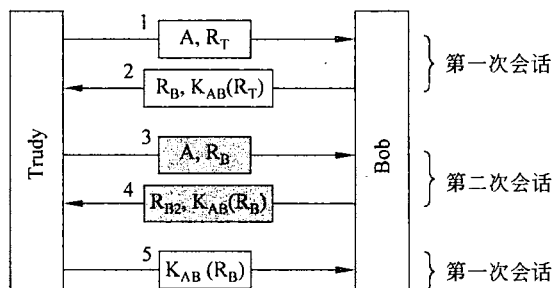


图 8-34 反射攻击

她可以利用消息 3 打开第二个会话，并且用消息 2 中的 R_B 作为她的质询。Bob 平静地对 R_B 进行加密，并在消息 4 中送回 $K_{AB}(R_B)$ 。我们为第二个会话上的消息加了阴影，以便突出显示它们。现在，Trudy 有了缺少的信息，所以她可以完成第一个会话，并放弃第二个会话。Bob 现在相信 Trudy 就是 Alice，所以当她想请求查询 Alice 的银行账户余额时，Bob 毫无怀疑地告诉了她。然后，她请求 Bob 将所有的余额转到瑞士银行的一个秘密账户中，他也毫不犹豫地照办不误。

这个故事的寓意是：

设计一个正确的认证协议要比表面看上去的复杂得多。

下面 4 条通用规则通常有助于协议设计者避免陷入常见的陷阱：

(1) 让发起方首先证明自己是谁，然后再轮到应答方。在上面的例子中，在 Trudy 给出证据来证明自己是谁之前 Bob 先给出了有价值的信息。

(2) 让发起方和应答方使用不同的密钥作为证明，不过，这意味着要使用两个共享密钥 K_{AB} 和 K'_{AB} 。

(3) 让发起方和应答方从不同的集合中选取质询随机数。例如，发起方必须使用偶数，而应答方必须使用奇数。

(4) 使协议能够抵抗这种涉及第二个并行会话的攻击，比如像上面给出的反射攻击那样，在一个会话中得到的信息可以用在另一个会话中。

只要违反了以上规则中的任何一条，则协议通常就会被攻破。在前面的例子中，所有 4 条都违反了，所以带来了灾难性的后果。

现在我们回去再仔细看图 8-32。这个协议真的不会遭受反射攻击吗？难说，那要看情

况了。情况很微妙。Trudy 能够用反射攻击来挫败我们的协议是因为她有可能打开第二个与 Bob 之间的会话，然后诱使他回答自己的问题。如果 Alice 是一台能够接受多个会话的通用计算机，而不是计算机前面的一个人，那会怎么样呢？现在我们来看看 Trudy 能做什么。

为了看清楚 Trudy 的攻击是如何实施的，请参考图 8-35。Alice 首先在消息 1 中宣告自己的标识。Trudy 截取了这条消息，并开始她自己的会话，声称自己是 Bob，如图中的消息 2 所示。同样地，我们用阴影来表示第二个会话的消息。Alice 在消息 3 中回应了消息 2，她这样说：你是 Bob？请证明这一点。这时候，Trudy 被定住了，因为她无法证明自己是 Bob。

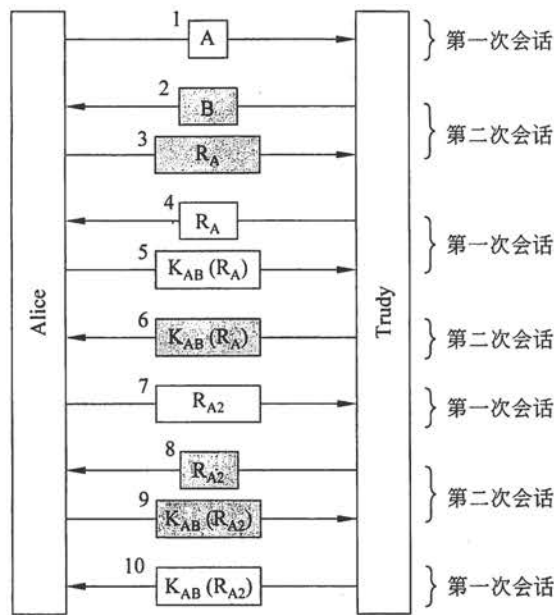


图 8-35 对图 8-32 所示的协议的反射攻击

现在 Trudy 该怎么办呢？她又回到第一个会话中，在这里，该轮到她发送一个质询，于是，她把在消息 3 中得到的 R_A 发送出去。Alice 非常友好地做了回应，即消息 5，从而给 Trudy 提供了她所急需的信息，所以，Trudy 发送了消息 6，此消息属于会话 2。此时，Trudy 基本上畅行无阻了，因为在会话 2 中，她已经成功地回应了 Alice 的质询。现在她可以取消会话 1 了，对于会话 2 的剩余部分，她发送任何一个老的数值，因此，她获得了一个与 Alice 之间经过认证的会话，即会话 2。

但是 Trudy 比我们想象的还要恶劣，她想彻底地攻进去并且不留下任何异常。她并不是马上发送一个老的数值来完成会话 2，而是等待 Alice 发送消息 7，即会话 1 中 Alice 的质询随机数。当然，Trudy 不知道该如何回应，所以她再次利用反射攻击，送回 R_{A2} ，即消息 8。Alice 很自然地加密了 R_{A2} ，并通过消息 9 送给 Trudy。Trudy 现在切换到会话 1，并且在消息 10 中把 Alice 想要的加密结果送给她，实际上她只是将 Alice 在消息 9 中发送过来的加密结果复制回去而已。到这时候，Trudy 与 Alice 之间有了两个完全经过认证的会话。

与图 8-34 中显示的针对三次消息协议的攻击相比较，上面的攻击有一个略微不同的结

果。这一次, Trudy 与 Alice 之间有了两个经过认证的连接。而在前一个例子中, 她与 Bob 之间只有一个经过认证的连接。同样在这里, 如果我们应用了前面讨论过的全部通用认证协议规则, 那么, 这种攻击就可以被制止。有关这些攻击以及如何阻止这些攻击的详细讨论, 请参考 (Bird 等, 1993)。他们还证明了: 完全有可能用系统化的方法来构造一个可被证明是正确的协议。然而, 对于这样的协议, 即使是最简单的形式也较为复杂, 所以, 我们现在来介绍他们提出的另外一类同样可以工作的协议。

新的认证协议如图 8-36 所示 (Bird 等, 1993)。它使用了 HMAC, 我们曾经在学习 IPsec 时了解过它。Alice 首先给 Bob 发送一个临时值 (nonce) R_A 作为消息 1。Bob 的回应是: 选择他自己的临时值 R_B , 并连同一个 HMAC 一起发送回去。这里的 HMAC 是这样形成的: 首先建立一个数据结构, 其中包含了 Alice 的临时值、Bob 的临时值、他们的标识, 以及共享密钥 K_{AB} ; 然后将这个数据结构做散列 (hash) 运算, 比如使用 SHA-1 算法, 散列的结果即为 HMAC。当 Alice 接收到消息 2 时, 她现在拥有 R_A (这是她自己选取的)、 R_B (这是以明文形式传来的)、双方的标识, 以及密钥 K_{AB} (这是她一直知道的), 所以她自己也可以计算出 HMAC。如果她计算的 HMAC 与消息中的 HMAC 一致, 那么她知道她在与 Bob 通话, 因为 Trudy 不知道 K_{AB} , 因此 Trudy 不可能计算出该发送哪一个 HMAC。Alice 回应给 Bob 的也是一个 HMAC, 但是此 HMAC 仅仅包含两个临时值和 K_{AB} 。

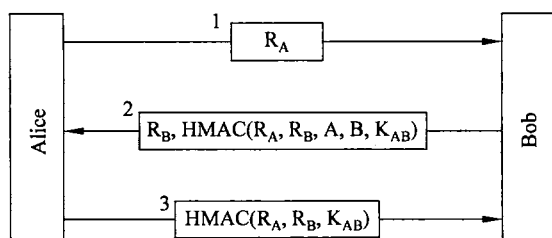


图 8-36 使用 HMCA 的认证

那么, Trudy 能够破坏这个协议吗? 不能, 因为她不可能像图 8-34 和图 8-35 中那样, 强迫任何一方按照她的选择来加密或者散列一个值。这里的两个 HMAC 中包含了双方选择的值, 其中部分值是 Trudy 所无法控制的。

采用 HMAC 并不是体现这种思想的唯一做法。除了在一组数据项上计算 HMAC 这种做法以外, 另一种常用的方案是利用密码块链 (cipher block chaining) 模式顺序地加密所有的数据项。

8.7.2 建立共享密钥: Diffie-Hellman 密钥交换

到现在为止, 我们一直假定 Alice 和 Bob 共享一个密钥。现在假设他们之间并没有共享密钥 (因为至今还没有一个被普遍接受的 PKI 可用于签名和分发证书)。他们如何才能建立一个共享密钥呢? 一种办法是, Alice 打电话给 Bob, 通过电话将她的密钥告诉 Bob, 但是 Bob 可能一上来就这样说: 我怎么知道你是 Alice 而不是 Trudy 呢? 他们可能会试着安排一次会面, 每个人都带上护照、驾驶证和三个主要的信用卡, 但如果他们都很忙碌的话, 他们有可能几个月都找不到一个双方都能接受的会面日期。幸运的是, 虽然听起来有

点难以置信,但是存在一种办法可以让完全陌生的人在完全公开的情况下建立起一个共享的密钥,即使 Trudy 小心地记录下每一条消息也无妨。

用在陌生人之间建立共享密钥的协议称为 **Diffie-Hellman 密钥交换协议** (Diffie-Hellman key exchange) (Diffie 和 Hellman, 1976), 其工作过程如下所述。Alice 和 Bob 必须就两个大数 n 和 g 达成一致, 这里 n 是一个素数, $(n-1)/2$ 也是一个素数, 并且 g 需要满足一些特殊的条件。这些数可以是公开的, 所以, 他们两人中的任何一个选取 n 和 g , 并且通过公开的方式告诉另一个人。现在 Alice 选择一个大数 x (比如说 1024 位), 并将它保密; 类似地, Bob 也秘密地选择一个大数 y 。

Alice 发起密钥交换协议, 她给 Bob 发送一条消息, 其中包含了 $(n, g, g^x \bmod n)$, 如图 8-37 所示。Bob 给 Alice 发送一条包含了 $(g^y \bmod n)$ 的消息作为应答。现在, Alice 把 Bob 发送给自己的数计算 x 次乘方并且以 n 为模, 得到 $(g^y \bmod n)^x \bmod n$ 。Bob 也执行类似的计算, 得到 $(g^x \bmod n)^y \bmod n$ 。根据模算术定理, 双方的计算结果都是 $g^{xy} \bmod n$ 。你瞧, 就像变魔术一样, Alice 和 Bob 突然之间共享了一个密钥, 即 $g^{xy} \bmod n$ 。

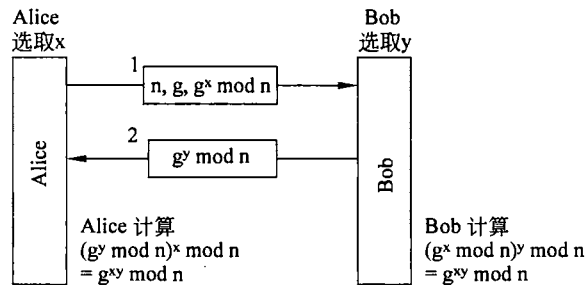


图 8-37 Diffie-Hellman 密钥交换

Trudy 当然也看到了这两条消息。她从消息 1 中知道了 g 和 n 。如果她能够计算出 x 和 y , 那么, 她就可以得到密钥。问题在于, 仅仅知道了 $g^x \bmod n$, 她不可能找到 x 。在以一个非常大的素数为模的情况下, 计算离散对数的实用算法目前尚未发现。

为了使上面的例子更加具体化, 我们假设 $n=47$ 和 $g=3$ (这些值完全是不切实际的)。Alice 选取 $x=8$, Bob 选取 $y=10$ 。这两个数值都是保密的。Alice 发给 Bob 的消息是 $(47, 3, 28)$, 因为 $3^8 \bmod 47$ 是 28。Bob 发给 Alice 的消息是 (17) 。Alice 计算 $17^8 \bmod 47$, 等于 4。Bob 计算 $28^{10} \bmod 47$, 也等于 4。所以, Alice 和 Bob 现在单独地确定了密钥为 4。为了找到密钥, Trudy 必须要解方程式: $3^x \bmod 47=28$, 对于像这里给出的数值, 利用穷举搜索可以解出 x 的值, 但是当所有这些数值都是几百位长时, 这种做法就行不通了。所有当前已知的算法都需要太长的时间才能解出 x , 即使在大型的并行超级计算机上也是如此。

尽管 Diffie-Hellman 算法非常优美, 但是它也存在一个问题: 当 Bob 得到了三元组 $(47, 3, 28)$ 时, 他怎么知道该消息来自 Alice 而不是 Trudy? 他没有办法判断这一点。不幸的是, Trudy 正好可以利用这个事实来同时欺骗 Alice 和 Bob, 过程如图 8-38 所示。这里, 在 Alice 和 Bob 分别选择了 x 和 y 时, Trudy 也选取了她自己的随机数 z 。Alice 发送消息 1 给 Bob, 但是 Trudy 截获了此消息, 并且发送消息 2 给 Bob; 在消息 2 中, Trudy 使用了正确的 g 和 n (不管怎么样, 这两个数都是公开的), 但是她用自己的 z 来代替 x 。她还将消息 3 送回给 Alice。后来, 当 Bob 给 Alice 发送消息 4 时, Trudy 再次截获此消息, 并保留

不转发。

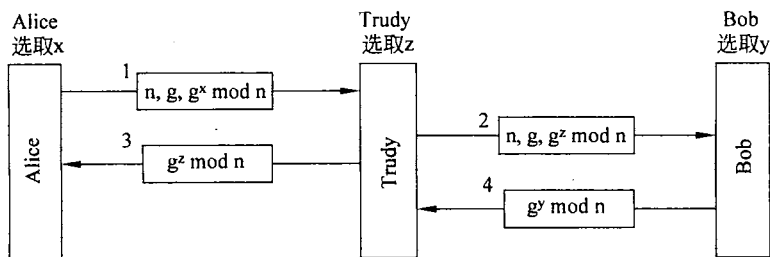


图 8-38 中间人攻击

现在，每个人都做模算术运算。Alice 计算出密钥为 $g^{xz} \bmod n$ ，Trudy 也一样（针对所有发送给 Alice 的消息）。Bob 计算出 $g^{yz} \bmod n$ ，Trudy 也做同样的计算（针对所有发送给 Bob 的消息）。Alice 认为她在跟 Bob 通话，所以她建立一个会话密钥（实际上，与 Trudy 共享此会话密钥）。Bob 也如此。Alice 在此加密会话上发送的所有消息都被 Trudy 捕获到，并存储下来，如果愿意的话做适当修改，然后（可选地）传递给 Bob。在另一个方向上也类似。Trudy 看到了所有的秘密，而且可以随意地改动所有的消息，而 Alice 和 Bob 都错误地认为自己与对方有一条安全的信道。基于这个原因，这种攻击也称为中间人攻击（man-in-the-middle attack），或者称为水桶队列攻击（bucket brigade attack），因为它有点类似于以前的义务消防队的做法，他们从救火车到着火点排成一队传递水桶。

8.7.3 使用密钥分发中心的认证

与陌生人建立共享密钥差不多可以工作了，但是并不完全解决问题。另外，有可能从一开始就不值得这么做（酸葡萄攻击）。若通过这种方式与 n 个人进行通话，那么你将需要 n 个密钥。对于普通人来说，密钥管理将变成一份实实在在的负担，尤其当每个密钥都必须被存储在一个单独的塑料芯片卡上的时候。

另一种不同的方法是引入一个可信的密钥分发中心（KDC）。在这种模型中，每个用户与 KDC 共享一个密钥。现在认证和会话密钥管理都通过 KDC 来完成。图 8-39 显示了目前已知的最简单的 KDC 认证协议，它涉及两方和一个可信的 KDC。

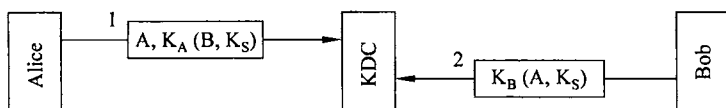


图 8-39 在认证协议中首次使用 KDC 的尝试

这个协议背后的思想非常简单：Alice 选取一个会话密钥 K_S ，并且告诉 KDC 她希望利用 K_S 与 Bob 进行通话。这条消息用 Alice 与 KDC 之间的共享密钥 K_A （只有 Alice 和 KDC 才知道）加密。KDC 解密此消息，并提取出 Bob 的标识和会话密钥。然后它构造一条新的消息，其中包含了 Alice 的标识和会话密钥，并且它将这条消息发送给 Bob。这次用 Bob 与 KDC 之间的共享密钥 K_B 加密。当 Bob 解密出该消息时，他就知道 Alice 想与他通话，并且她希望使用 K_S 作为会话密钥。

这里的认证是自然而然的。KDC 知道消息 1 一定来自 Alice，因为其他人不可能用 Alice

的密钥来加密消息。类似地，Bob 知道消息 2 一定来自于 KDC（而 Bob 对 KDC 是信任的），因为其他没有人知道他的密钥。

不幸的是，这个协议有一个严重的缺陷。Trudy 需要一些钱，所以她想出一些自己可以帮助 Alice 处理的合法业务，向 Alice 提出优厚的条件，最后得到了这份工作。在完成了工作以后，Trudy 礼貌地请求 Alice 通过银行转账来支付费用。然后 Alice 与她的银行服务员 Bob 建立一个会话密钥。之后她给 Bob 发送一条消息，请求将一笔钱转到 Trudy 的账户中。

同时，Trudy 又故伎重演，她在网络上窃听流量，并且将图 8-39 中的消息 2 和随后的转账请求消息复制下来。稍后，她又把这两条消息重新发送给 Bob。Bob 收到这两条重放消息后，他会这样想：Alice 一定又雇用了 Trudy，很显然 Trudy 干得不错。于是，Bob 又一次将同样数额的一笔钱从 Alice 的账户转到 Trudy 的账户上。经过了 50 次这样的消息重放以后，Bob 走出办公室并找到 Trudy，表示愿意为她提供一大笔贷款，以便 Trudy 能够拓展她那看起来显然很成功的业务。这个问题称为重放攻击（replay attack）。

针对重放攻击，有几种可能的解决方案。第一种方案是在每一条消息中包含一个时间戳。之后，如果有人接收到一条过期的消息，丢弃此消息即可。这种做法的麻烦之处在于，网络上的时钟从来都不是精确同步的，所以，总是存在某一个时间间隔使得在此期间时间戳是有效的。Trudy 就可以在这段时间间隔中重放消息，逃脱时间戳的检查。

第二种方案是在每条消息之中放置一个临时值。然后，每一方必须要记住所有此前已经出现过的临时值，如果一条消息包含了以前用过的临时值，那么拒绝该消息。但是，这些临时值必须被永久地记录下来，以免 Trudy 可能会试图重放 5 年前的消息。而且，如果某台机器崩溃之后丢失了所有的临时值信息，那么它又会再次遭受重放攻击。我们可以将时间戳和临时值结合起来，以便缩短需要记录临时值的时间范围，但是很显然这将会导致协议变得非常复杂。

一种较为复杂的实现双向认证的做法是使用多路径的质询-回应协议。这种协议的一个著名的例子是 Needham-Schroeder 认证协议（Needham 和 Schroeder, 1978），图 8-40 显示了它的一种变化形式。

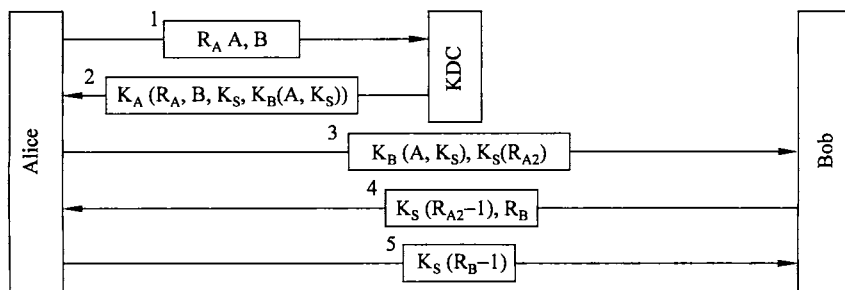


图 8-40 Needham-Schroeder 认证协议

协议开始时，Alice 首先告诉 KDC 她想与 Bob 通话。这条消息包含一个大的随机数 R_A 作为临时值。KDC 发回消息 2，其中包含了 Alice 的随机数、一个会话密钥，以及一个她将来可以发送给 Bob 的票据。这里使用随机数 R_A 是为了使 Alice 相信消息 2 是最新的，而不是重放消息。Bob 的标识也被包含在消息中，这是为了防止 Trudy 用她自己的标识来

替换消息 1 中的 B，从而导致 KDC 用 K_T 而不是 K_B 来加密消息 2 末尾的票据；经 K_B 加密的票据也被包含在加密的消息中，这是为了防止在返回到 Alice 的途中，Trudy 用其他的数据来偷换此票据。

Alice 现在将票据以及一个用会话密钥 K_S 加密的新随机数 R_{A2} 发送给 Bob。在消息 4 中，Bob 送回 $K_S(R_{A2}-1)$ ，以此向 Alice 证明，她在跟真正的 Bob 通话。如果直接送回 $K_S(R_{A2})$ 是没有用的，因为 Trudy 可以直接从消息 3 中偷到 $K_S(R_{A2})$ 。

Alice 在接收到消息 4 以后，她现在确信自己正在跟 Bob 通话，而且到现在为止没有任何重放消息。毕竟她仅仅在几毫秒之前才产生了 R_{A2} 。消息 5 的用途是为了使 Bob 确信，现在与他通话的是真正的 Alice，而且这里也没有出现任何重放消息。在这个协议中，每一方都生成一个质询，并且回应一个质询，因而消除了任何一种重放攻击的可能性。

尽管这个协议看起来似乎非常坚固，但它确实有一个微小的弱点。如果 Trudy 曾经设法获得过一个老的会话密钥的明文，那么她可以向 Bob 发起一个新的会话，具体做法是把对应于泄漏密钥的消息 3 重放给 Bob，使 Bob 相信她是真正的 Alice (Denning 和 Sacco, 1981)。现在，她甚至一次都不为 Alice 提供合法业务就可以抢劫她的银行账户了。

Needham 和 Schroeder 后来发表的一个协议改正了这个问题 (Needham 和 Schroeder, 1987)。在同一份杂志的同一期上，(Otway 和 Rees, 1987) 也发表了一个同样解决这个问题的协议，而且更加简短。图 8-41 显示了一个略作修改之后的 Otway-Rees 协议。

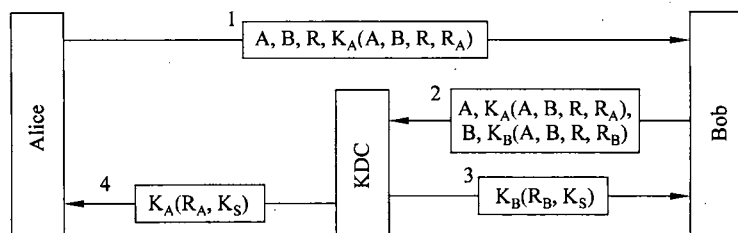


图 8-41 稍微被简化的 Otway-Rees 认证协议

在 Otway-Rees 协议中，Alice 首先生成一对随机数 R 和 R_A ，这里 R 被用作一个公共的标识符，而 R_A 则是 Alice 质询 Bob 的随机数。当 Bob 得到了这条消息时，他根据 Alice 消息中的加密部分构造一个新的消息，以及一条用他自己的信息构造出来的类似消息。用 K_A 和 K_B 加密的部分标识了 Alice 和 Bob，同时也包含了公共的标识符和一个质询随机数。

KDC 对这两部分中的 R 进行检查，看它们是否相同。它们有可能不相同，比如 Trudy 篡改了消息 1 中的 R，或者替换了消息 2 中的部分数据。如果两个 R 一致，则 KDC 相信 Bob 的请求消息是有效的。然后它生成一个会话密钥，并且对它加密两次，一次为 Alice，另一次为 Bob。在它发送给 Alice 和 Bob 的两条消息中，每条消息都包含了接收方的随机数，以此来证明这是 KDC 而不是 Trudy 生成的消息。这时候 Alice 和 Bob 拥有了同一个会话密钥，于是他们便可以开始通信了。当他们第一次交换数据消息时，每一方都看到对方有一个完全相同的 K_S ，所以认证过程便完成了。

8.7.4 使用 Kerberos 的身份认证

许多实际系统（包括 Windows 2000 以及后续版本）使用的认证协议是 Kerberos，它以

Needham-Schroeder 协议的一种变化形式为基础。Kerberos 得名于古希腊神话中一头专门守卫地狱大门的多头狗（大概它也被禁止离开地狱）。Kerberos 由 MIT 设计，设计目标是允许工作站用户以一种安全的方式访问网络资源。它与 Needham-Schroeder 协议的最主要不同之处在于，它假设所有的时钟都取得良好同步。Kerberos 协议经历了几次改版，其中第 5 版已被广泛应用于工业界，并且标准化为 RFC 4120。较早的第 4 版在出现严重问题后最终退役（Yu 等，2004）。第 5 版在第 4 版的基础上，对协议和功能方面做了很多小的改变，例如它再也不基于现在已经过时的 DES。有关更多的信息，请参看（Neuman 和 Ts'o, 1994）。

除了 Alice（一个客户工作站）以外，Kerberos 还涉及 3 个服务器：

- (1) 认证服务器（AS）：在用户登录过程中验证用户的身份。
- (2) 票据发放服务器（TGS, Ticket-Granting Server）：发放“身份票据的证明”。
- (3) Bob 服务器：真正完成 Alice 所请求的工作。

AS 与 KDC 非常类似，它与每个用户共享一个秘密口令。TGS 的任务是发放票据，这个票据可使真正的服务器相信：TGS 票据的持有者确实是他或她所声称的那一位。

为了开始一个会话，Alice 在任意一台公共的工作站前面坐下来，输入她的名字。该工作站将她的名字和 TGS 名字以明文方式发送给 AS，如图 8-42 中所示的消息 1。AS 发送回来的是一个会话密钥和一个替代 TGS 的票据 $K_{TGS}(A, K_S, t)$ 。会话密钥使用 Alice 的密钥加密，因此唯有 Alice 才能解密此消息。只有当消息 2 到达后，工作站才会请求 Alice 输入口令。然后工作站利用该口令生成 K_A ，来解密消息 2 获得会话密钥。

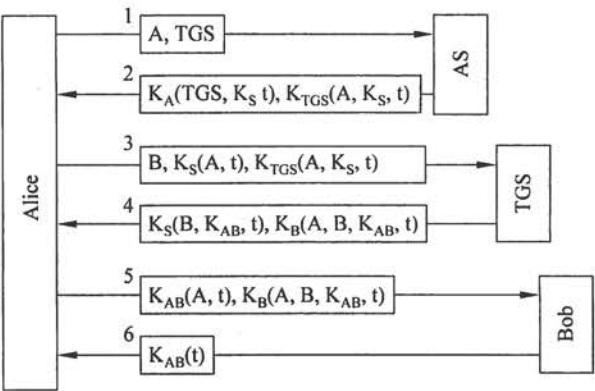


图 8-42 Kerberos 第 5 版的操作

这时候，工作站覆盖掉 Alice 的口令，以确保它只会在工作站里最多停留几毫秒的时间。如果 Trudy 试图以 Alice 的身份登录，那么，她输入的密码将是错误的，工作站将会检测到这种错误，因为消息 2 的标准部分是不正确的。

当 Alice 登录后，她可能告诉工作站，她想要联系 Bob 文件服务器。然后工作站给 TGS 发送消息 3，请求一个使用 Bob 文件服务器的票据。这个请求中的关键要素是 $K_{TGS}(A, K_S, t)$ ，它用 TGS 的密钥加密的，它的用途是证明发送方真的是 Alice。TGS 的回应是，创建并返回一个会话密钥 K_{AB} ，供 Alice 和 Bob 通信时使用。该会话密钥的两个版本都被发送回来，第一个版本仅仅用 K_S 加密，所以 Alice 可以读取它；第二个版本是另一个票据，用 Bob 的密钥 K_B 加密，所以 Bob 可以读取它。

Trudy 可以复制消息 3，并再次发送此消息，但是，她将会失败，因为连同消息 3 一起

发送的还有一个加密时间戳 t 。Trudy 不可能用一个更新的时间戳来代替 t ，因为她不知道 K_S ，即 Alice 用来与 TGS 通话的会话密钥。即使 Trudy 极为快速地重发了消息 3，她所得到的也只不过是消息 4 的另一份副本而已，既然第一次她不能解密消息 4，那么第二次她同样也不能解密。

现在 Alice 可以通过该新票据将 K_{AB} 发送给 Bob 以便与他建立一个会话（消息 5）。这次消息交换也使用了时间戳。可选的 Bob 的响应（消息 6）向 Alice 证明了，她确实是在与 Bob 而不是 Trudy 通话。

经过这一系列的消息交换以后，Alice 就可以在 K_{AB} 的保护下与 Bob 进行通信了。如果她后来决定要跟另一个服务器 Carol 进行通话，那么她只要向 TGS 再次发送消息 3，只不过这一次指定 C 而不是 B。TGS 将迅速地回送一个用 K_C 加密的票据，以后 Alice 可以将此票据发送给 Carol，而 Carol 则凭此票据相信她收到的消息确实来自 Alice。

所有这些工作的要点在于现在 Alice 可以以一种安全的方式来访问网络上所有的服务器了，而且她的口令永远都不会出现在网络上。事实上，即使在她自己的工作站上，口令也只是停留了几毫秒而已。然而，请注意，每个服务器都有它自己的授权规则。当 Alice 向 Bob 出示她的票据时，此票据仅仅向 Bob 证明了这是谁发送过来的。至于到底允许 Alice 做什么事情，则要取决于 Bob。

由于 Kerberos 的设计者们并不期望全世界都信任同一个认证服务器，所以他们提供了对多个域（realm）的支持，每个域有自己的 AS 和 TGS。为了得到一个针对远程域中某个服务器的票据，Alice 需要向她自己所在域的 TGS 请求一个票据，该票据必须能被远程域 TGS 接受。如果远程 TGS 已经向本地的 TGS 注册过了（其做法就如同本地服务器的注册方法一样），则本地 TGS 将给 Alice 一个可用于远程 TGS 的有效票据。然后她就可以在那边（即远程域）完成各种事务，比如获取针对该域中各个服务器的票据。然而，请注意，为了让两个域中的多个参与方共同完成一些工作，每个域必须信任另一个域的 TGS。否则，它们将无法工作。

8.7.5 使用公开密钥密码学的认证

可以利用公开密钥密码学来完成双向认证。首先，Alice 需要得到 Bob 的公钥。如果有一个 PKI 目录服务器可以提供公钥证书的查询服务，那么 Alice 可以请求 Bob 的公钥证书，如图 8-43 中的消息 1。消息 2 中的应答是一个 X.509 证书，其中包含了 Bob 的公钥。当 Alice 验证了证书中的签名无误后，她给 Bob 发送一条消息，该消息包含了她的标识和一个临时值。

当 Bob 接收到这条消息时，他无从判断这条消息到底来自于 Alice 还是 Trudy，但是他可以等待一下，并且向目录服务器请求 Alice 的公钥（消息 4），很快他便获得了 Alice 的公钥（消息 5）。然后，他给 Alice 发送一条消息，即图中的消息 6，其中包含了 Alice 的 R_A 、Bob 自己的临时值 R_B 和一个建议的会话密钥 K_S 。

当 Alice 得到了消息 6 以后，她用自己的私钥将它解密。她看到了里边的 R_A ，这让她有一种很温暖的感觉。这条消息一定是 Bob 发送过来的，因为 Trudy 无法确定 R_A 。而且，它也一定是最新的，而不是一条重放消息，因为她刚刚把 R_A 发送给 Bob。Alice 发回消息

7 表示她同意这次会话。当 Bob 看到了 R_B 是用他刚刚生成的会话密钥来加密时，他知道 Alice 已经得到了消息 6 并且验证了 R_A 。现在，Bob 心满意足了。

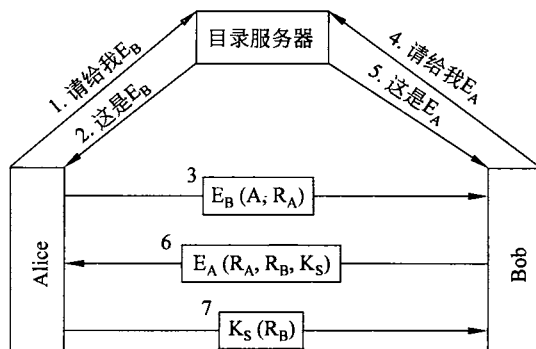


图 8-43 使用公开密钥密码学的双向认证

Trudy 怎样才能破坏这个协议呢？她可以虚构消息 3，以此来引诱 Bob 向 Alice 发送消息 6，但是 Alice 将会发现这个 R_A 不是她发送的，所以不会进行下去。Trudy 不可能伪造出 Alice 发送回给 Bob 的消息 7，因为她不知道 R_B 或者 K_S ，在没有 Alice 私钥的情况下，她是不可能确定 R_B 和 K_S 的。因此，面对这个协议时她的运气就太不好了。

8.8 电子邮件安全性

当电子邮件消息在两个远程站点之间传递时，一般情况下该消息沿途要经过几十台机器。这些机器中的任何一台都可以阅读和记录下该消息，以备将来可能之用。实际上，不管人们怎么想，隐私是不存在的。然而，许多人希望自己发送的电子邮件只有目标接收者才能阅读，其他人都无法阅读，即使他们的老板或者他们的政府也不能阅读。这种愿望刺激了一些人和组织将前面学习过的密码学原理应用到电子邮件上，从而形成安全的电子邮件。在本节中，我们将首先学习一个已经得到广泛应用的安全电子邮件系统：PGP，然后再简短介绍另外一个安全电子邮件系统 S/MIME。有关安全电子邮件的更多信息，请参看 (Kaufman 等, 2002) 和 (Schneier, 1995)。

8.8.1 PGP——良好的隐私性

我们的第一个例子良好的隐私性 (PGP, Pretty Good Privacy) 本质上是 Zimmermann 一个人的心血结晶 (Zimmermann, 1995a, 1995b)。他是一个极其崇尚隐私的人，他的人生格言是：“如果隐私是非法的，那么只有违法者才会有隐私”。PGP 发布于 1991 年，它是一个完整的电子邮件安全软件包，提供了私密性、认证、数字签名和压缩功能，而且所有这些功能都非常易于使用。此外，完整的软件包中也包含了所有的源代码，并通过 Internet 免费分发。由于 PGP 的质量、价格 (0)，以及在 UNIX、Linux、Windows 和 Mac OS 平台上的易用性，今天 PGP 已得到了广泛的使用。

PGP 使用一个称为国际数据加密算法 (IDEA, International Data Encryption Algorithm)

的块密码算法来加密数据。该算法使用 128 位密钥，它在瑞士被设计出来，正值 DES 被认为不够安全而 AES 尚未发明之际。概念上讲，IDEA 非常类似于 DES 和 AES：它也运行许多轮，在每一轮中将数据位尽可能地混合起来，但是，它的混合函数与 DES 和 AES 的不同。PGP 的密钥管理使用 RSA，数据完整性使用 MD5，这些话题我们前面都已经讨论过了。

PGP 自从诞生以来，就被卷入到一场持久的争论之中 (Levy, 1993)。因为 Zimmermann 并没有阻止其他人把 PGP 放到 Internet 上，全世界的任何人都可以得到 PGP，因此，美国政府声称 Zimmermann 违反了美国的军用武器出口法。美国政府对 Zimmermann 进行了长达 5 年的调查，但是最终放弃了，这里可能有两个原因。第一，Zimmermann 自己并没有把 PGP 放到 Internet 上，所以他的律师声称他没有出口任何东西（至于是否创建了一个出口物品的 Web 站点则是一个小问题了）。第二，政府最终意识到，要想赢得这个案子，意味着要使陪审团相信：一个可下载隐私程序的 Web 站点也应该被列入军火交易法的适用范围内，因而对 Web 站点的限制就如同禁止出口战争武器，比如坦克、潜艇、军用飞机和核武器。而且多年来的反面宣传可能没有多大帮助。

顺便提一下，将出口法应用在 PGP 上是非常不恰当的。美国政府认为把代码放在一个 Web 站点上是一种非法的出口，并为此侵扰了 Zimmermann 达 5 年之久。另外，如果某个人以一本书的形式发表 PGP 完整的 C 代码（采用比较大的字体，并且每一页带一个校验和，因此扫描识别非常容易），然后出口这本书，那么，政府认为这没有什么关系，因为图书并不是军品。剑利于笔，至少山姆大叔是这样认为的。

PGP 的另一个问题是它牵涉到了专利侵犯案。拥有 RSA 专利的公司是 RSA Security 公司，它宣称 PGP 使用 RSA 算法侵犯了它的专利，但是 2.6 以后的版本就不存在这个问题了。此外，PGP 还使用了另一个受专利保护的加密算法，即 IDEA，起初使用也曾经引起了一些问题。

由于 PGP 是源码开放的，所有的人和组织都可以对它进行修改，再产生各种版本。其中有些版本的设计目标是为了回避军品出口法，而其他有些版本则是为了避免使用专利算法，另外还有一些版本则是为了将它变成一个源码封闭的商业产品。虽然军品出口法现在已有所放宽（否则，使用 AES 的产品将不可能从美国出口），而且，RSA 的专利也已于 2000 年 9 月到期，但是，所有这些问题遗留下来的结果是有许多不兼容的 PGP 版本流传在各地，而且名字也不尽相同。下面的讨论将集中在经典的 PGP 上，这是最老的也是最简单的版本。另一个流行的版本是 OpenPGP，由 RFC 2440 描述。还有一个版本是 GNU Privacy Guard。

PGP 特意使用已有的密码学算法，而不是发明新的算法。它主要基于那些已经经受了广泛审查并且未受任何政府机构影响（政府机构总是试图削弱它们）的算法，对于那些不太信任政府的人来说，这个特性是一个很大的吸引点。

PGP 支持正文压缩、私密性和数字签名，也提供了可扩展的密钥管理工具，但奇怪的是，它没有提供电子邮件工具。它就像一个预处理器，接受明文输入，并产生签过名的密文作为输出，其输出格式为 Base64。然后这个输出被电子邮件程序发送出去。有些 PGP 实现的最后一步调用用户代理，以便真正将邮件发送出去。

为了看清楚 PGP 是如何工作的，我们考虑图 8-44 中的例子。这里，Alice 想要通过一

种安全的方式给 Bob 发送一个签名的明文邮件 P 。Alice 和 Bob 都有私有 (D_X) 和公开 (E_X) RSA 密钥。我们假设每一方都知道对方的公钥；我们将扼要地介绍 PGP 的密钥管理机制。

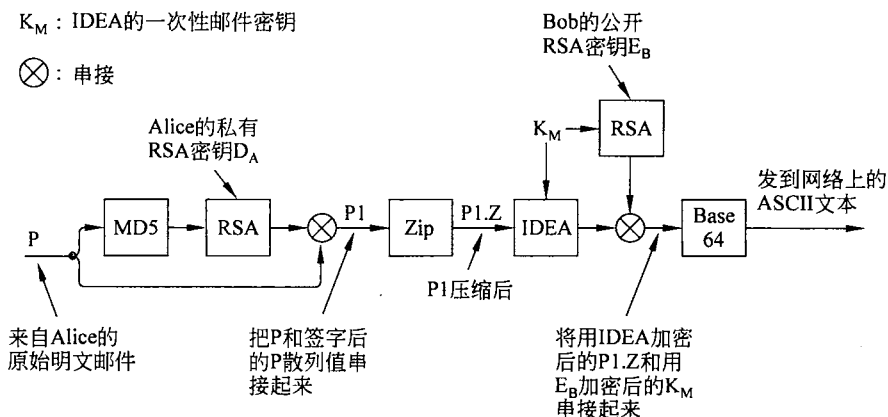


图 8-44 在发送消息的操作中的 PGP

作为开始, Alice 调用她自己机器上的 PGP 程序。PGP 首先使用 MD5 算法对她的消息 P 做散列运算, 然后用 Alice 的私有 RSA 密钥 D_A 加密此结果散列值。当 Bob 最终得到该消息时, 他可以用 Alice 的公钥解密此散列值, 并且验证散列值是否正确。即使此时其他人 (比如 Trudy) 也获得这个散列值, 并且也可以用 Alice 的公钥对它解密, 但 MD5 的强度保证了要产生另一个具有相同 MD5 散列值的消息在计算上是不可行的。

经过加密的散列值和原始的消息现在被串接成单条消息 $P1$ 中, 并且使用 ZIP 程序进行压缩。ZIP 程序使用了 Ziv-Lempel 算法 (Ziv 和 Lempel, 1977)。我们将这一步的输出结果称为 $P1.Z$ 。

接下来, PGP 提示 Alice 输入一些随机的信息。Alice 输入的内容和敲键的速度被用来生成一个 128 位的 IDEA 邮件密钥 K_M (在 PGP 的文献中该密钥称为会话密钥, 但是, 这实际上并不太恰当, 因为这里没有会话)。现在按照密码反馈模式, 利用 IDEA 算法和 K_M 来加密 $P1.Z$ 。另外, 使用 Bob 的公钥 E_B 来加密 K_M 。然后, 这两部分内容被串接起来, 并转换成 Base64 编码, 正如我们在第 7 章中讨论 MIME 时所介绍的那样。结果邮件只包含字母、数字和符号 +、/ 和 =, 这意味着该邮件可以被放到一个 RFC 822 邮件体中, 并且有望未被修改地到达另一方。

当 Bob 得到此邮件时, 他做一个逆向的 Base64 编码, 并且利用他自己的私有 RSA 密钥解密出 IDEA 密钥。利用这个密钥, 他解密此邮件得到 $P1.Z$; 再经过解压缩, Bob 将明文与加密的散列值分离开, 并且使用 Alice 的公钥解密此散列值。如果得到的明文散列值与他自己计算的 MD5 散列值一致, 那么, 他知道 P 是正确的邮件, 而且它确实来自于 Alice。

值得注意的是, 在这里只有两个地方用到了 RSA: 为了加密 128 位的 MD5 散列值, 以及加密 128 位 IDEA 密钥。虽然 RSA 非常慢, 但是它只需加密 256 位数据, 而不是大量的数据。而且, 所有这 256 位明文数据是高度随机的, 因此 Trudy 需要做相当大量的工作才能确定一个猜测出来的密钥是否正确。在 PGP 的操作过程中, 繁重的加密工作是由 IDEA 来完成的, 而 IDEA 比 RSA 快了几个数量级。因此, PGP 不仅提供了安全性、压缩和数字

签名的功能，而且它完成这些功能的效率比图 8-19 中显示的方案要高得多。

PGP 支持 4 种 RSA 密钥长度。它可以由用户选择一种最为合适的长度。这些长度选项是：

- (1) 临时的 (384 位)：今天很容易被破解。
- (2) 商用的 (512 位)：可被三字母组织破解。
- (3) 军用的 (1024 位)：地球上的任何人都无法破解。
- (4) 星际的 (2048 位)：其他行星上的任何人也无法破解。

由于 RSA 仅仅被用在两个小的计算过程中，所以，每个人在任何时候都应该使用星际强度的密钥。

经典 PGP 消息的格式如图 8-45 所示。在实际中还使用了其他许多格式。图中显示的消息有 3 个部分，分别包含了 IDEA 密钥、签名和消息体。密钥部分包含的不仅仅是密钥，还有一个密钥标识符，因为 PGP 允许每个用户有多个公钥。

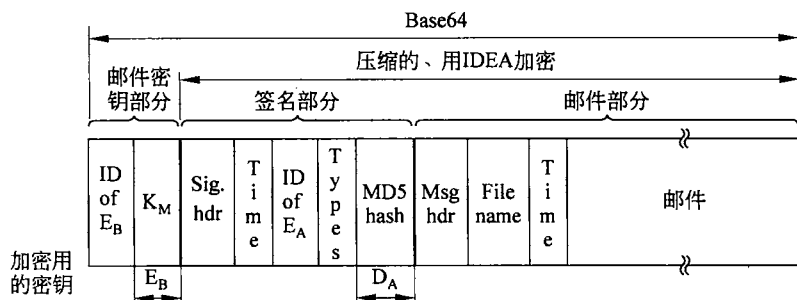


图 8-45 一个 PGP 邮件

签名部分包含一个头，这里我们并不关心这个头。头的后面紧跟一个时间戳、发送方公钥的标识符、某些类型信息以及加密的散列值本身。其中，公钥标识符指定了用来解密签名散列值的公钥，而类型信息则标识了所使用的算法（当 MD6 和 RSA2 发明出来时可以直接支持它们）。

邮件部分也包含一个头，以及一个默认的文件名，如果接收方要将文件写到磁盘中的话，则可以使用此默认的文件名。另外，邮件部分还包含了一个用来标识该邮件被创建时刻的时间戳，最后是邮件本身。

在 PGP 中，密钥管理部分最为受到关注，因为它是所有安全系统的要害所在。PGP 的密钥管理模型如下所述。每个用户在本地维护两个数据结构：一个私钥环和一个公钥环。**私钥环** (private key ring) 包含一个或者多个本人的“公钥/私钥”对。为每个用户支持多对“公钥/私钥”的原因是允许用户定期地改变他们的公钥，或者当用户认为某一个私钥已被泄漏时，他无须将当前已准备好或者正在传送过程中的邮件统统作废就可以改变公钥。每一对“公钥/私钥”都有一个标识符与其关联，所以邮件发送方可以告诉接收者自己是使用哪一个公钥来对它加密的。该标识符由公钥的低 64 位构成的。避免公钥标识符发生冲突是用户的责任。磁盘上的私钥必须使用特殊的口令（任意长）来加密，以避免它们被偷窥。

公钥环 (public key ring) 包含了与当前用户通信的其他用户的公钥。为了加密与每个邮件相关联的邮件密钥，这些公钥是必要的。公钥环上的每个条目不仅包含了公钥，而且

也包含它的 64 位标识符，以及一个代表用户信任此密钥的强烈程度的指示值。

这里需要解决的问题如下所述。假设公钥被存放在公告板上。Trudy 为了阅读 Bob 的秘密电子邮件，一种办法是攻击公告板，并且用她自己选择的公钥来代替 Bob 的公钥。当 Alice 后来获得了这个自称属于 Bob 的公钥时，Trudy 就可以对 Bob 实施水桶队列攻击。

为了防止这样的攻击，或者至少将攻击的后果减到最小，Alice 需要知道在她的公钥环中，她对这个自称是“Bob 的公钥”有多大的信任程度。如果她知道该公钥来自于 Bob 亲自交给她的软盘，那么她就可以将信任值设置到最大。正是这种非中心化的、由用户控制的公钥管理方法使得 PGP 无须采用中心化的 PKI 方案。

然而，有时候人们还是要通过查询一个可信的密钥服务器来获得其他用户的公钥。由于这个原因，在 X.509 被标准化以后，PGP 除了支持传统的 PGP 公钥环机制以外，也支持 X.509 证书。所有当前的 PGP 版本都支持 X.509。

8.8.2 S/MIME

IETF 冒险进军电子邮件安全领域的结果是安全的 MIME (S/MIME, Secure/MIME)，该协议由 RFC 2632~2643 描述。它提供了认证、数据完整性、保密性和不可否认性。而且，它还非常灵活，支持大量的密码学算法。无须惊奇的是既然使用了这样的名字，那么它一定与 MIME 集成得很好，从而可以保护各种类型的邮件。此外，它也定义了许多新的 MIME 头，比如，用来存放数字签名的 MIME 头。

S/MIME 并没有一个严格的、从单个根开始的证书层次结构，因为这种结构早已成为早期系统增强保密邮件 (PEM, Privacy Enhanced Mail) 的政治问题之一。相反，用户可以有多个信任锚。只要一个证书能够被回溯到当前用户所相信的某个信任锚，它就被认为是有效的。S/MIME 使用了标准的算法和协议，关于这些算法和协议，我们前面都已经学习过了，所以这里不再进一步讨论它们。有关 S/MIME 的细节，请参考相应的 RFC。

8.9 Web 安全性

我们前面已经学习了两个非常需要安全性的重要领域：通信和电子邮件。你可以将它们想象成饭前汤和开胃小吃。现在该上主菜了：即 Web 安全。Web 是如今的大多数 Trudy 们的主要工作场所，正是在 Web 上他们做着卑劣的事情。在本节，我们将讨论与 Web 安全有关的一些问题和事项。

我们可以粗略地将 Web 安全分成 3 个部分。第一，如何安全地命名对象和资源？第二，如何建立安全的、真实的连接？第三，当一个 Web 站点向客户发送一段可执行代码时，会发生什么？下面我们首先看一些潜在的威胁，之后我们再详细地讨论每一部分。

8.9.1 威胁

人们几乎每个星期都可以在报纸上看到有关 Web 站点安全的问题。这种状况确实非常严酷。我们现在看一些已经发生过的例子。首先，许多组织的主页受到过攻击，被代之以

骇客(cracker)选择的新主页(流行的出版物将这些侵入别人计算机的人称为黑客(hacker),但是许多程序员把“黑客”这个术语留给了那些超级程序员。本书中我们将这些人称为“骇客”)。曾经遭受过这种破坏的站点包括 Yahoo、美国军方、CIA、NASA 和纽约时报。在绝大多数情况下,骇客只是放置一些稀奇古怪或者调侃的文字,因而遭受破坏的站点可以在几小时内得到修复。

现在我们看一些更加严重的案例。有许多站点曾经因为遭受拒绝服务攻击而停止运行,在这些攻击中,骇客用大量的流量淹没掉 Web 站点,使它不能对合法的请求做出响应。通常这种攻击来自大量已被骇客攻破(DDoS 攻击)的机器。这样的攻击非常常见,所以它们甚至已经不再是新闻了,但是,它们却可以使被攻击的站点造成成千上万美元的损失。

1999 年,一名瑞典的骇客侵入了 Microsoft 的 Hotmail Web 站点,并创建了一个镜像站点,该镜像站点允许任何人输入 Hotmail 用户的名字,然后读取该用户当前的和已经存档的所有电子邮件。

在另一个案例中,一个名叫 Maxim 的 19 岁俄罗斯骇客侵入了一个电子商务 Web 站点,并且偷到了 300 000 个信用卡号码。然后,他联系上站点的所有者们,并告诉他们如果他们不付给他 100 000 美元,他将把所有的信用卡号码张贴到 Internet 上。他们并没有屈从于他的敲诈勒索,结果他真的张贴了信用卡号码,极大地损害了许多无辜者的利益。

在另一个不同类型的事件中,一名 23 岁的加利福尼亚学生以电子邮件的形式给一家通讯社发了一则新闻稿,谎称 Emulex 公司将要发布有关季度亏损的消息,而且 CEO 也要立即辞职。在几小时之内,该公司的股票暴跌了 60%,从而导致股票持有者损失了 20 亿美元以上。而作案者在发送公告之前不久卖出了他的股票,他赚了 25 万美元。虽然这起事件不属于 Web 站点入侵,但是很显然,这样的公告放在任何一家大公司的主页上,也会有类似的效果。

我们可以用许多页的篇幅来介绍类似这样的事件(这真的很不幸)。但现在该介绍与 Web 安全相关的技术要素了。有关各种安全问题的更多信息,请参考(Anderson, 2008)、(Stuttard 和 Pinto, 2007)以及(Schneier, 2004)。你也可以在 Internet 上搜索到大量的实际案例。

8.9.2 安全命名

我们首先从一些最基本的问题入手: Alice 希望访问 Bob 的 Web 站点。她在她的浏览器中敲入 Bob 的 URL,几秒钟以后,一个 Web 页面出现了。但是,这真的是 Bob 的主页吗?可能是,也可能不是。Trudy 可能又重施故技了。例如,她可能截获了 Alice 发出来的所有数据包,并查看了这些数据包的内容。当她捕捉到有一个 HTTP GET 请求指向 Bob 的 Web 站点时,她可以到 Bob 的 Web 站点上将页面取过来,然后根据她的意愿进行修改,再将修改之后的假页面返回给 Alice。Alice 对此一无所知。更糟的是,Trudy 可以大幅降低 Bob 电子商场中的货物价格,以便使他的货物看起来非常有诱惑力,从而诱使 Alice 将她的信用卡号码发送给 Bob 以购买商品。

这种经典的中间人攻击的一个缺点是,Trudy 必须位于某一个能够截取到 Alice 的出境

流量以及能够伪造她入境流量的位置上。实际上, 她必须搭接在 Alice 的电话线上, 或者 Bob 的电话线上, 因为搭接到光纤骨干线路上相当困难。虽然主动搭线这种方法肯定行得通, 但是它需要一定的工作量; 另外, 虽然 Trudy 非常聪明, 但同时她也非常懒惰。除了这种做法以外, 还有其他更加容易的方法在等着 Alice 去尝试。

DNS 欺骗

Trudy 能够攻破 DNS 系统的一种方式或许是 Alice 的 ISP 上的 DNS 缓存, Trudy 用她自己的 IP 地址 (比如说 42.9.9.9) 来替换 Bob 的 IP 地址 (比如说 36.1.2.3)。这将会导致下述的攻击发生。图 8-46(a)显示了设想中的工作过程。这里, (1) Alice 向 DNS 请求 Bob 的 IP 地址; (2) 得到了地址; (3) 向 Bob 请求他的主页; (4) 得到了 Bob 的主页。当 Trudy 将 Bob 的 DNS 记录修改成包含她自己的 IP 地址而不再是 Bob 的地址时, 我们得到了如图 8-46(b)所示的情形。这里, 当 Alice 查询 Bob 的 IP 地址时, 她得到了 Trudy 的 IP 地址, 所以她期望发送给 Bob 的流量全部转向 Trudy。现在 Trudy 就可以实施中间人攻击, 而无须陷入到搭接任何电话线的麻烦之中。不过, 她必须要侵入一台 DNS 服务器, 并改变其中一条记录, 这可能会容易得多。

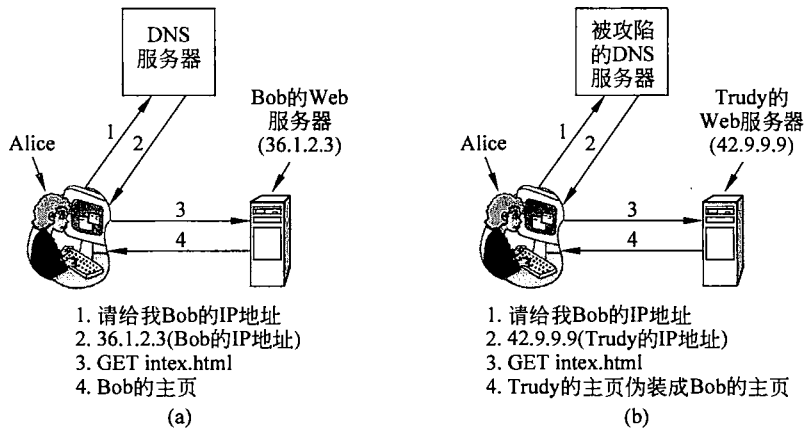


图 8-46

(a) 正常情况下; (b) 一个基于攻入 DNS 服务器和修改 Bob 记录的攻击

Trudy 如何欺骗 DNS 呢? 这实际上比较简单。简而言之, Trudy 可以诱使 Alice 的 ISP 的 DNS 服务器发出一个查询请求, 请求查找 Bob 的 IP 地址。由于 DNS 使用的是 UDP, 所以对于 DNS 服务器来说, 非常不幸的是, 它实际上并没有办法来检查到底谁提供了答案。Trudy 可以充分利用这种特性, 她的做法是, 伪造一个预设的应答数据包, 从而将一个假的 IP 地址插入到 DNS 服务器的缓存中。为了简化起见, 我们假设 Alice 的 ISP 初始时并不包含与 Bob 的 Web 站点 bob.com 相对应的 DNS 表项。如果已经存在的话, 则 Trudy 可以等待一段时间, 直到该表项超时然后再重试 (或者使用其他的技巧)。

Trudy 发起攻击的做法: 她向 Alice 的 ISP 发送一个查询请求, 请求 bob.com 的 IP 地址。由于不存在与这个 DNS 名字相对应的表项, 所以缓存服务器询问顶级 com 域服务器, 以便得到此 DNS 名字的 IP 地址。然而, Trudy 抢在 com 服务器之前先发送回一个假的应答: “bob.com 是 42.9.9.9”, 这里的 IP 地址正是她自己的地址。如果她的假应答首先到达

Alice 的 ISP 处, 则她的 IP 地址将被缓存起来, 而真正的应答则变成一个不请自来的应答而遭到拒绝。诱使 DNS 服务器安装一个假 IP 地址的技术称为 **DNS 欺骗 (DNS spoofing)**。像这样存放了一个刻意的假 IP 地址的缓存则称为**染毒缓存 (poisoned cache)**。

实际上, 事情并没有这么简单。首先, Alice 的 ISP 要检查该应答是否包含顶级服务器的正确 IP 源地址。但是, 由于 Trudy 可以在这个 IP 域中设置任何数值, 所以她很容易挫败这项测试, 因为顶级服务器的 IP 地址是公开的。

其次, 为了允许 DNS 服务器辨别哪个应答对应于哪个请求, 所有的请求都携带一个序号。为了欺骗 Alice 的 ISP, Trudy 必须知道它的正确序号。要想了解当前的序号, 对于 Trudy 来说, 最简单的办法是自己注册一个域, 比如说 `trudy-the-intruder.com`。我们假定它的 IP 地址也是 42.9.9.9。她为这个新谋划的域创建一个 DNS 服务器: `dns.trudy-the-intruder.com`。而且, 该服务器仍然使用 Trudy 的 IP 地址 42.9.9.9, 因为 Trudy 只有一台计算机。现在, 她必须让 Alice 的 ISP 知道她的 DNS 服务器。这很容易做到, 她只需向 Alice 的 ISP 请求 `foobar.trudy-the-intruder.com` 即可, 这会导致 Alice 的 ISP 询问顶级 com 服务器, 谁在为 Trudy 的新域提供 DNS 服务。

随着 `dns.trudy-the-intruder.com` 安全地进入到 Alice 的 ISP 的缓存中, 真正的攻击开始了。现在 Trudy 向 Alice 的 ISP 请求查询 `www.trudy-the-intruder.com`。该 ISP 很自然地 toward Trudy 的 DNS 服务器发送一个同样的查询请求。该请求携带的序号正是 Trudy 所努力寻找的。Trudy 高兴得跳了起来, 她立即请求 Alice 的 ISP 查询 Bob 的地址, 并迅速地发送一个伪造的应答来回答自己的问题, 而且谎称该应答来自顶级 com 服务器, 该应答的内容是: “`bob.com` 是 42.9.9.9”。这个伪造的应答中所携带的序号, 比她刚刚接收到的请求中的序号大 1。虽然她已经算得很准确了, 但是她也可以再发送第二个伪造的应答, 其中序号比刚才接收到的大 2, 甚至可能发送几十个伪造的应答, 其中的序号逐渐增加。这些应答中总有一个会匹配到 Alice 的 ISP 所发送的 DNS 查询请求, 剩下的将被丢弃。当这个伪造的应答到达时, 它被缓存起来; 而当真正的应答稍后到达时, 却遭到拒绝, 因为那时已经没有正在等待回应的查询了。

现在, 当 Alice 查询 `bob.com` 时, 他被告知应该使用 42.9.9.9, 即 Trudy 的地址。Trudy 在她自己舒适的客厅里成功地实施了中间人攻击。图 8-47 显示了这种攻击的各个步骤。这种特定的攻击可以通过让 DNS 服务器在发出的查询中使用随机 ID 而不是仅仅计数值来挫败, 但是看起来每当补上一个漏洞, 另一个又出现。特别是, ID 仅是 16 位, 所以用计算机很容易尝试所有的可能数字。

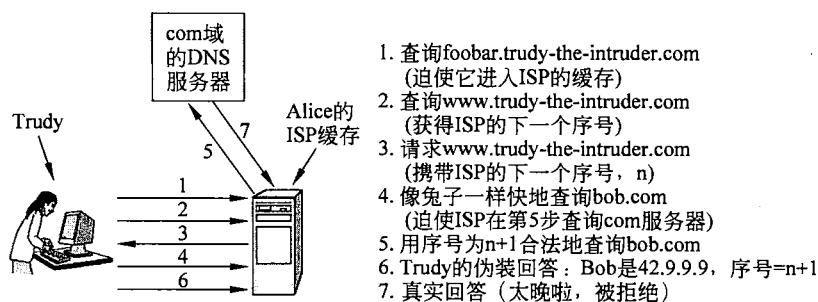


图 8-47 Trudy 如何欺骗 Alice 的 ISP

安全的 DNS

前面介绍的攻击很容易被挫败，只要让 DNS 服务器在它们发出的查询请求中使用随机的 ID 而不是按顺序递增的 ID。但事情总是这样，每当补上一个漏洞时，另一个漏洞又冒了出来。真正的问题在于，在 DNS 被设计时，Internet 只是一个仅供几百所大学使用的研究工具，而不是让 Alice、Bob 或者 Trudy 都加入进来的聚会场所。那时候安全并不是一个问题，使 Internet 能真正工作起来才最为重要。几十年来，网络环境已经有了很大的变化，所以，1994 年 IETF 成立了一个工作组致力于使 DNS 从本质上变得更加安全。这个（正在进行中）项目称为 DNS 安全（DNSsec, DNS Security）；它的第一项成果发表在 RFC 2535。不幸的是，DNSsec 尚未得到完全的部署，所以，目前有大量的 DNS 服务器仍然会遭受各种欺骗攻击。

从概念来看，DNSsec 极其简单。它建立在公开密钥密码学的基础之上。每一个 DNS 区域（按照图 7-5 中的含义）有一个“公钥/私钥”对。DNS 服务器发送的所有信息都经过签名，签名所用的私钥是发起区域的私钥，所以接收方可以验证它的真实性。

DNSsec 提供了 3 个基本服务：

- (1) 证明数据是从哪里发出来的。
- (2) 公钥的分发。
- (3) 事务和请求的认证。

主要的服务是第一个，它证实了返回的数据确实经过了区域所有者的同意。第二个服务对于安全地保存和获取公钥非常有用。第三个服务对于预防重放和欺骗攻击非常必要。请注意，DNSsec 并没有提供保密性服务，因为 DNS 中的所有信息都应该是公开的。按照预定的期望，DNSsec 需要几年时间才能被全部署，所以，具有安全功能的 DNS 服务器与不支持安全功能的服务器之间的协同工作能力是最基本的，这意味着 DNSsec 不能改变协议。现在我们来考察其中的一些细节。

DNS 的记录被组织成一些称为资源记录集（RRSet, Resource Record Sets）的集合，所有具有同样名字、类别和类型的记录被集中到一个集合中。一个 RRSet 可能包含多个 A 记录，例如，倘若一个 DNS 名字被解析到一个主要的 IP 地址和一个次要的 IP 地址。RRSet 也允许扩展新的记录类型（后面将讨论到）。每个 RRSet 都有一个密码学意义上的散列值（比如使用 MD5 或者 SHA-1），然后又利用该区域的私钥对散列值进行签名（比如使用 RSA 算法）。发送给客户的传输单元是经过签名的 RRSet。当客户接收到一个签过名的 RRSet 时，它可以验证此 RRSet 是否被发起区域签过名。如果签名没有问题，则可以接受 RRSet 中的数据。由于每个 RRSet 都包含它自己的签名，所以 RRSet 可以被缓存在任何地方，即使被缓存在不可信的服务器上也不会危及安全性。

DNSsec 引入了几种新的记录类型。第一种是 KEY 记录。这种记录包含了某一个区域、用户、主机或者其他主要的公钥，以及用于签名的密码算法、所使用的传输协议，还有其他一些位。这里的公钥是裸存的，DNSsec 没有使用 X.509 证书，因为它太庞大。若算法字段被设置为 1，则代表了签名方案使用 MD5/RSA 算法组合（首选的方案），其他的算法组合使用其他的值。协议字段表明了使用 IPSec 或者其他的安全协议（如果有的话）。

第二种新的记录类型是 SIG 记录。它存放的是经过签名的散列值，其中所用的算法由

KEY 记录来指定。这个签名作用在该 RRSset 中的所有记录上（即签名的范围），包括所有出现的 KEY 记录，但是不包括 SIG 记录自身。SIG 记录也包含了该签名有效周期的开始时间和失效时间、签名者的名字以及其他一些信息。

在 DNSsec 的设计方案中，每个区域的私钥可以一直保持离线状态。每天一次或者两次将一个区域的数据库通过手工方式（比如通过 CD-ROM）传输到一台未联网的机器上，这台机器上保存着私钥。所有的 RRsset 在那里被签名，由此而产生的 SIG 记录被传回到该区域的主服务器中。通过这种方式，私钥可以被保存在 CD-ROM 上，除了每天签名新的 RRsset 时需要将 CD-ROM 插入到未联网的机器中以外，其他时间该 CD-ROM 被锁在保险箱里。当签名完成以后，内存中和磁盘上所有的私钥副本都被擦除掉，并且 CD-ROM 又被放回保险箱里。这个过程实际上把电子安全还原为物理安全，人们对于物理安全总是比较容易理解和处理。

这种预先为 RRsset 做签名的方法大大加速了 DNS 服务器响应客户查询的过程，因为这样就不需要在处理过程中做密码学运算了。所付出的代价是，在 DNS 数据库中需要大量的磁盘空间来存储所有的密钥和签名。由于引入了签名，有些记录的长度将增加十倍以上。

当客户进程获得了一个签过名的 RRsset 时，它必须利用发起区域的公钥来解密散列值，同时自己计算一遍散列值，然后比较这两个值。如果它们相等，则认为此 RRsset 有效。然而，这个过程又引出了另一个问题，即客户如何得到一个区域的公钥。一种方法是利用一个安全的连接（比如使用 IPSec）通过一台可信的服务器来获得区域的公钥。

然而，实际上，DNSsec 假定客户已经预先配置了所有顶级域的公钥。如果 Alice 现在想要访问 Bob 的 Web 站点，则她可以向 DNS 请求 bob.com 的 RRsset，该 RRsset 中包含了 Bob 的 IP 地址和一个 KEY 记录（其中包含了 Bob 的公钥）。此 RRsset 已被 com 顶级域签过名，所以 Alice 很容易验证它的有效性。图 8-48 中显示了这个 RRsset 可能包含的内容。

域名	生存期	类别	类型	值
bob.com.	86 400	IN	A	36.1.2.3
bob.com.	86 400	IN	KEY	3682793A7B73F731029CE2737D...
bob.com.	86 400	IN	SIG	86947503A8B848F5272E53930C...

图 8-48 bob.com 的 RRsset 例子

KEY 记录是 Bob 的公钥。SIG 记录是顶级 com 服务器签过名的 A 和 KEY 记录的散列值，以便验证它们的真实性

现在 Alice 拥有了 Bob 公钥的一份副本，并且已经通过了验证，于是她向 Bob 的 DNS 服务器（由 Bob 运行）请求 www.bob.com 的 IP 地址。Bob 返回的 RRsset 将用 Bob 的私钥做签名，所以 Alice 可以验证这个 RRsset 的签名。如果 Trudy 设法将一个假的 RRsset 插入到任何一个缓存中，则 Alice 很容易就可以检测出它是不真实的，因为 RRsset 中的 SIG 记录将是不正确的。

然而，DNSsec 还提供了一种密码学机制把应答和特定的查询绑定起来，从而防止像图 8-47 中 Trudy 所做的那种欺骗。这种（可选的）反欺骗手段的做法是，应答方利用自己的私钥对查询消息的散列值做签名，然后把签名后的散列值也加入到应答消息中。由于 Trudy 不知道顶级 com 服务器的私钥，所以她不可能为 Alice 的 ISP 所发送的查询伪造一个应答。她当然可以让她的应答先返回到 Alice 的 ISP 处，但是它将遭到拒绝，因为她对查

询的散列值所做的签名是无效的。

DNSsec 也支持其他一些记录类型。例如，CERT 记录可以被用来保存证书（比如 X.509 证书）。之所以提供这个记录是因为有些人希望将 DNS 变成一个 PKI。这种变化是否真的会发生还有待于进一步观察。我们对于 DNSsec 的讨论到此为止，有关更多的细节，请参考 RFC 2535。

8.9.3 SSL——安全套接层

安全的命名机制是一个很好的起点，但是，对于 Web 安全而言还需要做得更多。下一步是安全的连接。我们现在考察如何实现安全的连接。安全涉及的事情绝不简单，这不是非此即彼。

当 Web 最初出现在公众面前时，它仅仅被用来发布静态页面而已。然而，不久之后，有些公司就想到了将 Web 用于金融交易，比如用信用卡购物、在线银行和电子股票交易。这些应用创造了一种新的需求，即迫切需要安全的连接。1995 年，Netscape 公司作为当时占主导地位的浏览器厂商，对此迅速做出了响应，他们引入了一个称为安全套接字层（SSL，Secure Sockets Layer）的安全软件包来迎合这种需求。这个软件包和它的协议现在已被广泛使用，甚至也为 Firefox、Safari 和 Internet Explorer 所采用，因此值得我们考察它的某些细节。

SSL 在两个套接字之间建立一个安全的连接，其中包括以下功能：

- （1）客户与服务器之间的参数协商。
- （2）客户和服务器的双向认证。
- （3）保密的通信。
- （4）数据完整性保护。

以前我们曾经看到过这些功能的确切含义，所以这里不再细致地加以阐述。

图 8-49 显示了 SSL 在通常协议栈中的位置。实际上，它位于应用层和传输层之间的一个新层，它接受来自浏览器的请求，再将请求传递给 TCP 以便传输到服务器上。一旦安全的连接已经被建立起来，则 SSL 的主要任务是处理压缩和加密。在 SSL 之上使用的 HTTP 尽管也是标准的 HTTP 协议，但是它称为安全的 HTTP（HTTPS，Secure HTTP）。有时候它使用一个新的端口（443），而不是标准的端口（80）。顺便提一下，尽管 SSL 并不仅限于被用在 Web 浏览器中，但这仍然是它最常见的应用。它也可以提供双向认证。

应用层(HTTP)
安全层(SSL)
传输层(TCP)
网络层(IP)
数据链路层(PPP)
物理层(调制解调器、ADSL、有线电视)

图 8-49 家庭用户使用 SSL 层（和协议）浏览网页

SSL 协议经历了几个版本。下面我们只讨论第 3 版，这是最为广泛使用的版本。SSL 支持许多不同的算法和选项。这些选项包括是否使用压缩功能、使用哪些密码学算法，以及一些与密码产品出口限制有关的事项。最后一项的意图是为了确保高强度的密码学技

术仅用于连接双方都在美国国内的情形。在其他情况下，密钥被限制在 40 位，许多密码学家对此非常不屑一顾。为了从美国政府这里拿到出口许可，Netscape 被迫加入了这项限制。

SSL 由两个子协议组成，一个用来建立安全的连接，另一个使用安全的连接。我们首先来看如何建立安全连接。图 8-50 显示了连接建立子协议。子协议从消息 1 开始，此时 Alice 向 Bob 发送一个建立连接的请求。该请求指定了 Alice 所用的 SSL 版本，以及她优先选择的压缩算法和密码学算法。它也包含一个临时值 R_A ，后面将会用到此值。

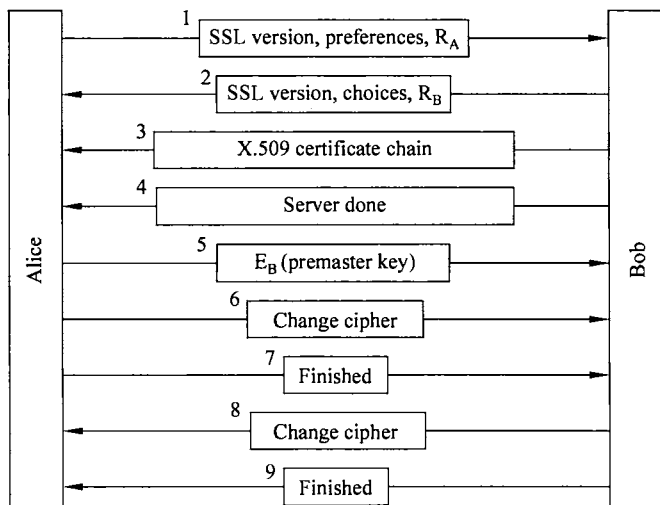


图 8-50 简化版的一个 SSL 连接建立子协议

现在轮到 Bob 了。在消息 2 中，Bob 在 Alice 可能支持的各种算法之中做出选择，并且将他的临时值 R_B 发送给 Alice。然后，在消息 3 中，他发送一个证书，其中包含了他的公钥。如果这个证书并没有被某一个知名的权威机构签过名，那么，他还发送一个证书链，通过这条证书链可以回溯到某一个由权威机构签名的证书。所有的浏览器，包括 Alice 的浏览器，都预装了大约 100 多个公钥，所以，如果 Bob 能够建立起一条证书链，并且此证书链以这 100 多个证书中的某一个作为信任锚，那么，Alice 将能够验证 Bob 的公钥。这时 Bob 可以发送其他一些消息（比如请求 Alice 的公钥证书）。当 Bob 完成以后，他发送消息 4 告诉 Alice 该轮到她了。

Alice 的响应是，选择一个随机的 384 位预设主密钥（premaster key），并且用 Bob 的公钥加密之后发送给 Bob（消息 5）。用于加密数据的实际会话密钥是从这个预设主密钥和两个临时值，通过一种复杂的方法推导得来的。当 Bob 接收到消息 5 以后，Alice 和 Bob 都能够计算会话密钥。由于这个原因，Alice 告诉 Bob，现在请切换到新的密码（消息 6），并且告诉他她已经完成了建立连接子协议（消息 7）。然后，Bob 对这两条消息分别进行确认（消息 8 和 9）。

然而，虽然 Alice 知道了 Bob 是谁，但是，Bob 并不知道 Alice 是谁（除非 Alice 有一个公钥和相应的证书，而这对于普通个人用户来说是不太可能的）。因此，Bob 的第一条消息极有可能是一个登录请求，请求 Alice 用一个预先建立的名字和口令登录进来。然而，此登录协议完全超出了 SSL 的范围。一旦 SSL 的安全连接已经建立起来，不管怎么样，数据传输就可以开始了。

正如上面所提到的，SSL 支持多种密码学算法。最强的一种方案使用三个独立密钥的三重 DES 来加密数据，通过 SHA-1 保护数据完整性。这种做法相对比较慢，所以它主要用于银行和其他一些对安全性要求比较高的应用。对于普通的电子商务应用，则使用 128 位密钥的 RC4 算法完成数据加密，通过 MD5 实现消息认证。RC4 将这个 128 位的密钥用作一个种子，并且将它扩展成一个非常大的数以便内部使用。然后，它利用这个内部数来生成一个密钥流。该密钥流与明文做异或（XOR）操作，从而提供了一个经典的流密码算法，正如我们在图 8-14 中看到的那样。出口版本也使用了 128 位密钥的 RC4 算法，但是其中 88 位是公开的，以使该密码算法易于被破解。

为了实际传输数据，我们需要使用第二个子协议，如图 8-51 所示。来自浏览器的消息首先被分割成最多 16 KB 的单元。如果当前连接支持压缩功能的话，则每个单元被单独压缩。之后，根据两个临时值和预设主密钥推导出来的一个密钥被串接到压缩之后的文本中，然后再利用已经协商好的散列算法（通常是 MD5）对串接之后的结果做散列运算。这个散列值被附加在每一个分段的尾部，作为它的消息认证码（MAC）。然后，再使用协商好的对称加密算法（通常是与 RC4 的密钥流进行 XOR 运算）对压缩之后的分段和 MAC 进行加密。最后，每个分段被附上一个分段头，再通过 TCP 连接被传送出去。

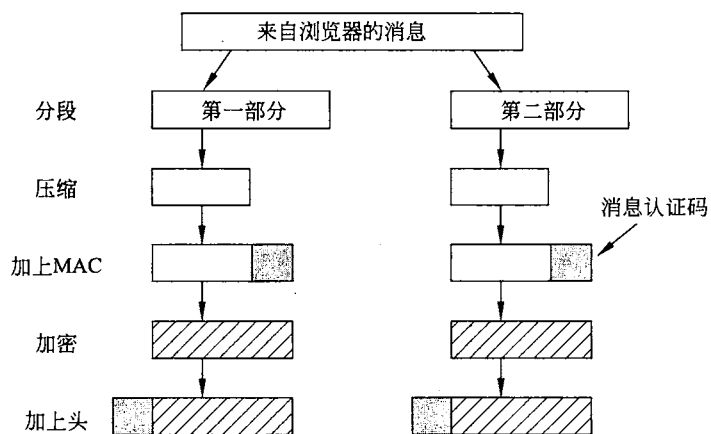


图 8-51 使用 SSL 的数据传输

然而，在这里有个忠告值得一提。由于研究人员已经证明了 RC4 算法存在一些弱密钥，即很容易被分析出密钥，所以使用 RC4 的 SSL 其安全性将是不可靠的（Fluhrer 等，2001）。如果浏览器允许用户选择密码算法组合，则用户应该将它配置成使用 168 位密钥的三重 DES 和 SHA-1，尽管这种组合比 RC4 和 MD5 要慢一些。

SSL 存在一个问题，即主角可能没有证书，或者即使他们有证书，他们也并不总是验证所用的密钥是否与证书相匹配。

1996 年，Netscape 公司将 SSL 移交给 IETF 进行标准化。最终的结果是传输层安全（TLS，Transport Layer Security）。RFC 5246 描述了 TLS。

TLS 基于 SSL 的第 3 个版本。尽管 IETF 对 SSL 所做的修改非常小，但即使这些小小的变化也足以使 SSL 版本 3 和 TLS 无法实现互操作。例如，从预设主密钥和临时值推导出会话密钥的方法被改变了，从而使得密钥更强大（即难以进行密码分析）。由于这种不兼容性，大多数浏览器都实现了两个协议，需要时可以协商将 TLS 回退到 SSL，这就是所谓的